

EDA Notebook

0. Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
12.9/12.9 MB 47.2 MB/s eta
0:00:00
```

```
4.9/4.9 MB 56.9 MB/s eta
0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

Mounted at /content/gdrive

You can now save your data files in:
/content/gdrive/MyDrive/36106/assignment/AT3/data

Student Information

```
# <Student to fill this section>
group_name = "Group 20"
student_name = "Ratnadeep Patra"
student_id = "26294153"

# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

```
<IPython.core.display.HTML object>

# Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)

<IPython.core.display.HTML object>

# Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)

<IPython.core.display.HTML object>
```

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

```
# <Student to fill this section>
```

0.b Import Packages

```
# <Student to fill this section>
import pandas as pd
import altair as alt

import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import os
```

B. Data Understanding

```
# Do not modify this code
try:
    df = pd.read_csv(at.folder_path / "product.csv")
except Exception as e:
    print(e)
```

B.1 Explore Dataset

```
pd.set_option('display.max_columns', None) # Forcing pandas to display
all columns
pd.set_option('display.max_rows', None) # Forcing pandas to display
all rows
```

```
df.describe(include='all').T
```

```
{
  "summary": {
    "name": "df",
    "rows": 23,
    "fields": [
      {
        "column": "count",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 886,
          "num_unique_values": 14,
          "samples": [
            497,
            381,
            886
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "unique",
        "properties": {
          "dtype": "date",
          "min": 1,
          "max": 504,
          "num_unique_values": 10,
          "samples": [
            4,
            37,
            18
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "top",
        "properties": {
          "dtype": "string",
          "num_unique_values": 13,
          "samples": [
            "2008-04-30 00:00:00",
            "L",
            "3615d396-f30f-4805-b326-de5b780a5507"
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "freq",
        "properties": {
          "dtype": "date",
          "min": "6",
          "max": 360,
          "num_unique_values": 10,
          "samples": [
            360,
            74,
            190
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "mean",
        "properties": {
          "dtype": "date",
          "min": 0.5067720090293454,
          "max": 530.8939051918736,
          "num_unique_values": 9,
          "samples": [
            448.01442437923254,
            0.5981941309255079,
            37.45283018867924
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "std",
        "properties": {
          "dtype": "date",
          "min": 0.4905400121432858,
          "max": 761.9530428102138,
          "num_unique_values": 9,
          "samples": [
            761.9530428102138,
            0.4905400121432858,
            34.339936373851124
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "min",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 4.0,
          "num_unique_values": 5,
          "samples": [
            4.0,
            2.12,
            3.0
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "25%",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 100.0,
          "num_unique_values": 5,
          "samples": [
            100.0,
            75.0,
            2.88
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "50%",
        "properties": {
          "dtype": "date",
          "min": 1.0,
          "max": 500.0,
          "num_unique_values": 7,
          "samples": [
            1.0,
            53.99,
            500.0
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "75%",
        "properties": {
          "dtype": "date",
          "min": 1.0,
          "max": 500.0,
          "num_unique_values": 7,
          "samples": [
            1.0,
            53.99,
            500.0
          ],
          "semantic_type": ""
        }
      },
      {
        "column": "max",
        "properties": {
          "dtype": "date",
          "min": 1.0,
          "max": 500.0,
          "num_unique_values": 7,
          "samples": [
            1.0,
            53.99,
            500.0
          ],
          "semantic_type": ""
        }
      }
    ]
  }
}
```

```

\ "75%", \n      \ "properties\ ": { \n      \ "dtype\ ": \ "date\ ", \n
\ "min\ ": 1.0, \n      \ "max\ ": 1000.0, \n
\ "num_unique_values\ ": 7, \n      \ "samples\ ": [ \n      1.0, \n
1000.0, \n      594.83 \n      ], \n      \ "semantic_type\ ":
\ "\ ", \n      \ "description\ ": \ "\ " \n      } \n      }, \n      { \n
\ "column\ ": \ "max\ ", \n      \ "properties\ ": { \n      \ "dtype\ ":
\ "date\ ", \n      \ "min\ ": 1.0, \n      \ "max\ ": 3578.27, \n
\ "num_unique_values\ ": 8, \n      \ "samples\ ": [ \n      1000.0, \n
2171.2942, \n      1.0 \n      ], \n      \ "semantic_type\ ":
\ "\ ", \n      \ "description\ ": \ "\ " \n      } \n      } \n      ] \n
n }", "type": "dataframe"}

```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 886 entries, 0 to 885
```

```
Data columns (total 23 columns):
```

#	Column	Non-Null Count	Dtype
0	product_id	886 non-null	object
1	is_manufactured	886 non-null	float64
2	is_sellable	886 non-null	float64
3	safety_stock_level	886 non-null	float64
4	reorder_point	886 non-null	float64
5	days_to_manufacture	886 non-null	float64
6	product_subcategory_id	530 non-null	object
7	product_model_id	530 non-null	float64
8	name	637 non-null	object
9	product_number	782 non-null	object
10	color	461 non-null	object
11	standard_cost	754 non-null	float64
12	list_price	886 non-null	float64
13	size	378 non-null	object
14	size_unit_measure_code	321 non-null	object
15	weight_unit_measure_code	376 non-null	object
16	weight	376 non-null	float64
17	product_line	497 non-null	object
18	class	453 non-null	object
19	style	381 non-null	object
20	sell_start_date	886 non-null	object
21	sell_end_date	166 non-null	object
22	discontinued_date	0 non-null	float64

```
dtypes: float64(10), object(13)
```

```
memory usage: 159.3+ KB
```

```
df.sample(10)
```

```
{"type": "dataframe"}
```

```
df.duplicated().sum()
```

```

np.int64(382)

df.drop_duplicates(inplace=True)

df.describe(include='all').T

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 23,\n  \"fields\": [\n    {\n      \"column\": \"count\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 0.0,\n        \"max\": 504,\n        \"num_unique_values\": 13,\n        \"samples\": [\n          \"98\",\n          \"278\",\n          \"504\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"unique\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 1,\n        \"max\": 504,\n        \"num_unique_values\": 10,\n        \"samples\": [\n          4,\n          37,\n          18\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"top\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 13,\n        \"samples\": [\n          \"2008-04-30 00:00:00\",\n          \"L\",\n          \"c62cc174-c497-4da7-8d2a-5a5068c9d246\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"freq\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"1\",\n        \"max\": \"211\",\n        \"num_unique_values\": 9,\n        \"samples\": [\n          \"211\",\n          \"43\",\n          \"100\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"mean\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 0.4742063492063492,\n        \"max\": 535.1507936507936,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          438.66624999999993,\n          0.5853174603174603,\n          37.44406779661017\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"std\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 0.49315664605418646,\n        \"max\": 773.6028426304881,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          773.6028426304881,\n          0.49315664605418646,\n          34.025442017484266\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"min\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 0.0,\n        \"max\": 4.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          2.12,\n          3.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"25%\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 0.0,\n        \"max\": 100.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          100.0,\n          2.88,\n          75.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"50%\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": 0.0,\n        \"max\": 100.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          100.0,\n          2.88,\n          75.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}"

```

```

{"min": 0.0, "max": 500.0, "num_unique_values": 8, "samples": [1.0, 23.3722], "semantic_type": "", "description": "", "column": "75%", "properties": {"dtype": "date", "min": 1.0, "max": 1000.0, "num_unique_values": 7, "samples": [1.0, 1000.0, 564.99], "semantic_type": "", "description": "", "column": "max", "properties": {"dtype": "date", "min": 1.0, "max": 3578.27, "num_unique_values": 8, "samples": [1000.0, 2171.2942, 1.0], "semantic_type": ""}, "description": ""}
n}, "type": "dataframe"}

```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 504 entries, 0 to 882
```

```
Data columns (total 23 columns):
```

#	Column	Non-Null Count	Dtype
0	product_id	504 non-null	object
1	is_manufactured	504 non-null	float64
2	is_sellable	504 non-null	float64
3	safety_stock_level	504 non-null	float64
4	reorder_point	504 non-null	float64
5	days_to_manufacture	504 non-null	float64
6	product_subcategory_id	295 non-null	object
7	product_model_id	295 non-null	float64
8	name	405 non-null	object
9	product_number	462 non-null	object
10	color	256 non-null	object
11	standard_cost	452 non-null	float64
12	list_price	504 non-null	float64
13	size	211 non-null	object
14	size_unit_measure_code	176 non-null	object
15	weight_unit_measure_code	205 non-null	object
16	weight	205 non-null	float64
17	product_line	278 non-null	object
18	class	247 non-null	object
19	style	211 non-null	object
20	sell_start_date	504 non-null	object
21	sell_end_date	98 non-null	object
22	discontinued_date	0 non-null	float64

```
dtypes: float64(10), object(13)
```

```
memory usage: 94.5+ KB
```

```
pd.to_datetime(df['sell_start_date'], errors='coerce').isnull().sum()
```

```

np.int64(0)

(df['sell_end_date'].isna().sum() -
 pd.to_datetime(df['sell_end_date'], errors='coerce').isnull().sum())

# This feature already have a lot of NA values, so those NA values are
ignored for date value analysis.

np.int64(0)

# <Student to fill this section>
dataset_insights = """
This product dataset initially contained 886 samples. The 'product_id'
feature, intended as a primary key, exhibited 504 unique values,
suggesting the
presence of duplicate entries. Subsequent analysis confirmed 382
duplicated samples, which can be dropped to ensure data integrity. The
dataset is
characterized by a significant extent of missing data across various
features. Specifically, 'is_manufactured', 'is_sellable',
'safety_stock_level',
'reorder_point', 'days_to_manufacture', 'list_price', and
'sell_start_date' are complete, while all other features contain
missing values. Notably, the
'discontinued_date' column is entirely devoid of entries. Furthermore,
the 'standard_cost' and 'size' features, despite being inherently
numeric, are
assigned an object data type, indicating potential data issues and
contain non-numeric characters within these columns. The
'sell_start_date' and
'sell_end_date' columns, upon validation, were found to contain valid
date formats.
"""

# Do not modify this code
print_tile(size="h3", key='dataset_insights', value=dataset_insights)

<IPython.core.display.HTML object>

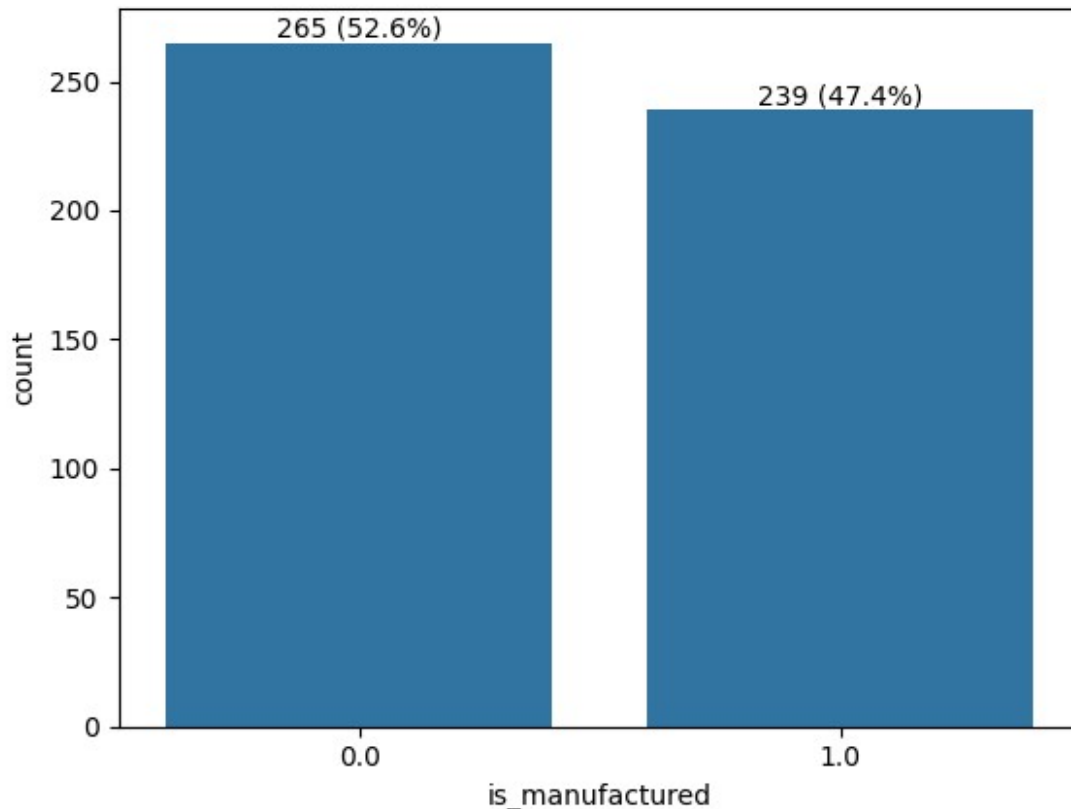
```

B.2 Explore Feature of Interest is_manufactured

```

sns.barplot(x=df['is_manufactured'].value_counts().index,
y=df['is_manufactured'].value_counts())
total = len(df['is_manufactured'])
for p in plt.gca().patches:
    percentage = '{:.1f}%'.format(100 * p.get_height()/total)
    x = p.get_x() + p.get_width() / 2
    y = p.get_height()
    plt.gca().annotate(f'{int(p.get_height())} ({percentage})', (x,
y), ha='center', va='bottom')

```



```
# <Student to fill this section>
```

```
feature_1_insights = """
```

```
The 'is_manufactured' feature is a binary categorical variable that indicates whether a product is manufactured in-house (represented by 1.0) or sourced
```

```
externally (represented by 0.0). The distribution of this feature is relatively balanced, with 239 products (47.4%) being manufactured in-house and 265
```

```
products (52.6%) being external. This balanced representation suggests that both manufacturing and sourcing strategies play significant roles in the
```

```
product catalog and can be further analyzed for potential correlations with other product attributes.
```

```
"""
```

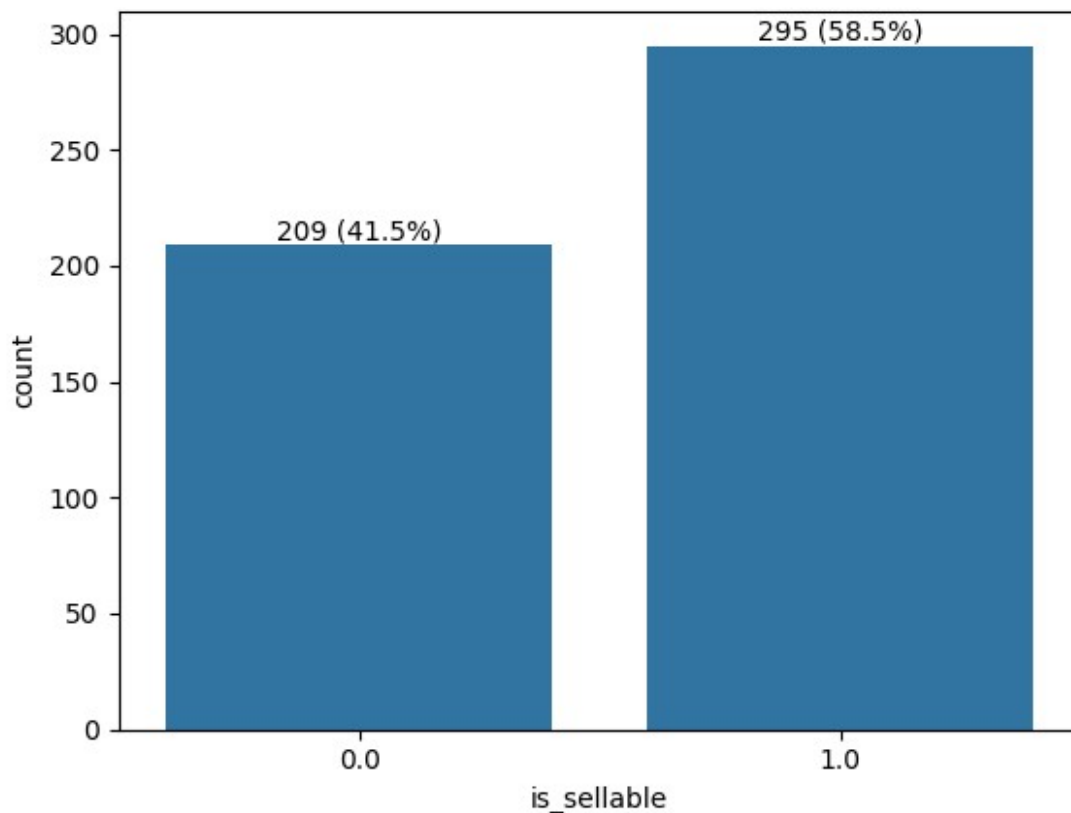
```
# Do not modify this code
```

```
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

```
<IPython.core.display.HTML object>
```

B.3 Explore Feature of Interest `is_sellable`

```
sns.barplot(x=df['is_sellable'].value_counts().index,  
y=df['is_sellable'].value_counts())  
total = len(df['is_sellable'])  
for p in plt.gca().patches:  
    percentage = '{:.1f}%'.format(100 * p.get_height()/total)  
    x = p.get_x() + p.get_width() / 2  
    y = p.get_height()  
    plt.gca().annotate(f'{int(p.get_height())} ({percentage})', (x,  
y), ha='center', va='bottom')
```



```
df.groupby('is_sellable')['is_manufactured'].value_counts()
```

```
is_sellable  is_manufactured  
0.0          0.0             182  
            1.0              27  
1.0          1.0             212  
            0.0              83
```

Name: count, dtype: int64

```
for is_sellable in [0, 1]:  
    for is_manufactured in [0, 1]:  
        print(f"is_sellable = {is_sellable} & is_manufactured =
```

```
{is_manufactured}")
    print(list(df[(df['is_sellable'] == is_sellable)
& (df['is_manufactured'] == is_manufactured)
& df['name'].notna()]).sample(3, random_state=42)['name']))

is_sellable = 0 & is_manufactured = 0
['Lock Washer 10', 'Thin-Jam Lock Nut 12', 'HL Road Rim']
is_sellable = 0 & is_manufactured = 1
['ML Touring Seat Assembly', 'Touring End Caps', 'Road End Caps']
is_sellable = 1 & is_manufactured = 0
['Hitch Rack - 4-Bike', 'Touring Tire', 'Bike Wash - Dissolver']
is_sellable = 1 & is_manufactured = 1
['Touring-2000 Blue, 46', 'Touring Rear Wheel', 'LL Touring Frame -
Yellow, 50']

# <Student to fill this section>
feature_2_insights = """
The 'is_sellable' feature is a binary categorical variable indicating
whether a product is available for sale (1.0) or not (0.0). Among the
504 unique
products in the dataset, 295 products (approximately 58.5%) are marked
as sellable, while 209 products (approximately 41.5%) are not. This
distribution
shows that a majority of products are intended for sale, but a
significant portion is not. When cross-referencing 'is_manufactured'
with 'is_sellable',
the products fall into four distinct business categories. Firstly,
Purchased & Not Sold (182 items) - These can be considered raw
materials. These include
products like road rim and lock nut. Secondly, we have Manufactured &
Sold (212 items) - These might be the core products. The company
builds these
in-house, and they are finished goods ready for the retail market.
These include products like seat assembly and end caps. Thirdly we
have Purchased & Sold
(83 items) - These are resale merchandise. The company buys these from
external vendors and sells them directly to the public. These include
products like
bike wash and tire. Lastly, we have Manufactured & Not Sold (27 items)
- These are internal sub-assemblies. The company builds these in-
house, but they are
not offered to the public. These include products like rear wheel and
bike frame.
"""

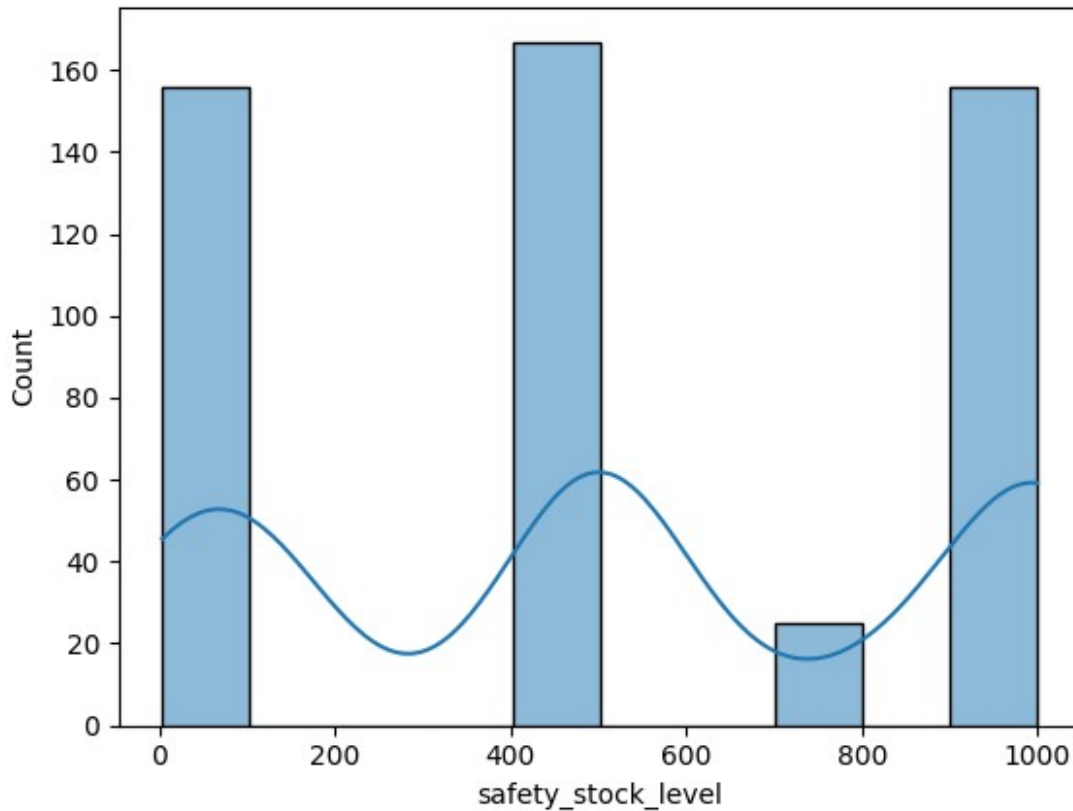
# Do not modify this code
print_tile(size="h3", key='feature_2_insights',
value=feature_2_insights)

<IPython.core.display.HTML object>
```

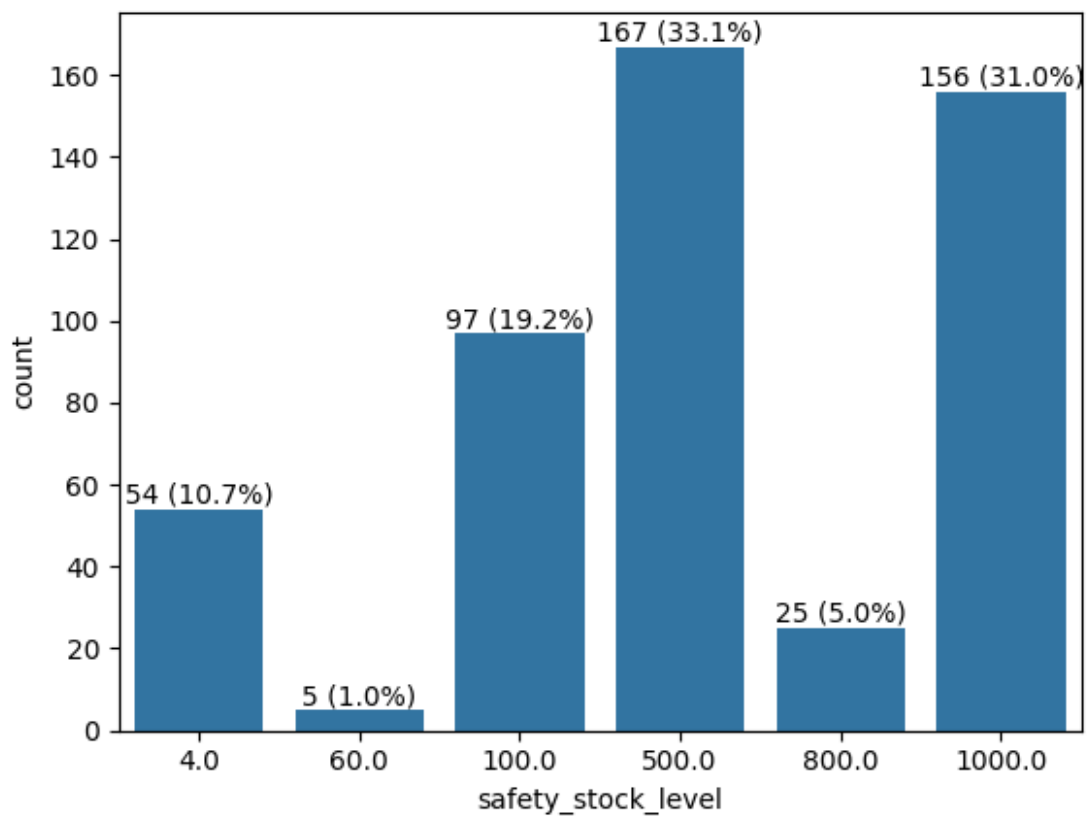
B.4 Explore Feature of Interest `safety_stock_level`

```
sns.histplot(df['safety_stock_level'], bins='auto', kde=True)
```

```
<Axes: xlabel='safety_stock_level', ylabel='Count'>
```

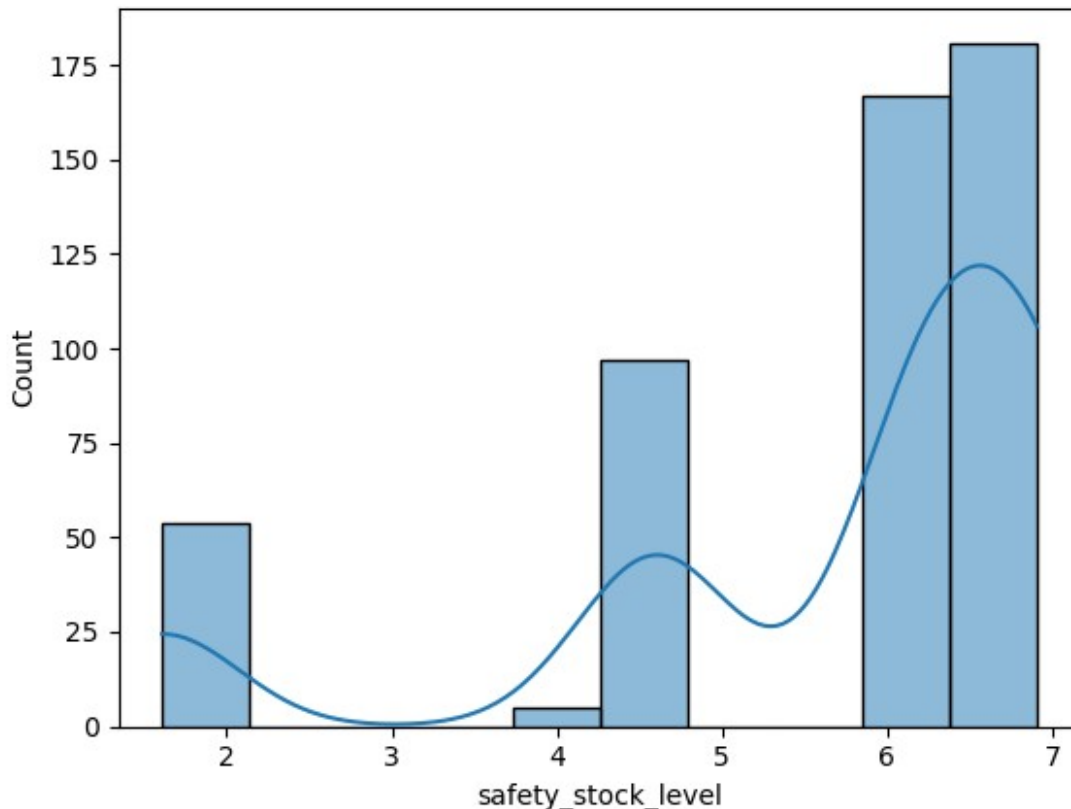


```
sns.barplot(x=df['safety_stock_level'].value_counts().index,  
y=df['safety_stock_level'].value_counts())  
total = len(df['safety_stock_level'])  
for p in plt.gca().patches:  
    percentage = '{:.1f}%'.format(100 * p.get_height()/total)  
    x = p.get_x() + p.get_width() / 2  
    y = p.get_height()  
    plt.gca().annotate(f'{int(p.get_height())} ({percentage})', (x,  
y), ha='center', va='bottom')
```



```
sns.histplot(np.log1p(df['safety_stock_level']), kde=True,  
bins='auto')
```

```
<Axes: xlabel='safety_stock_level', ylabel='Count'>
```



```
# <Student to fill this section>
feature_3_insights = """
The 'safety_stock_level' feature represents the safety stock quantity
for each product. While numerically continuous, this feature exhibits
a limited
number of distinct values (6 unique values) ranging from 4 to 1000.
Due to this a log transformation might be beneficial for future
analysis, to bring all
the values in a comparable scale. The distinct values suggest that
safety stock levels might be assigned based on specific product
categories or business
rules rather than being purely continuous.
"""
```

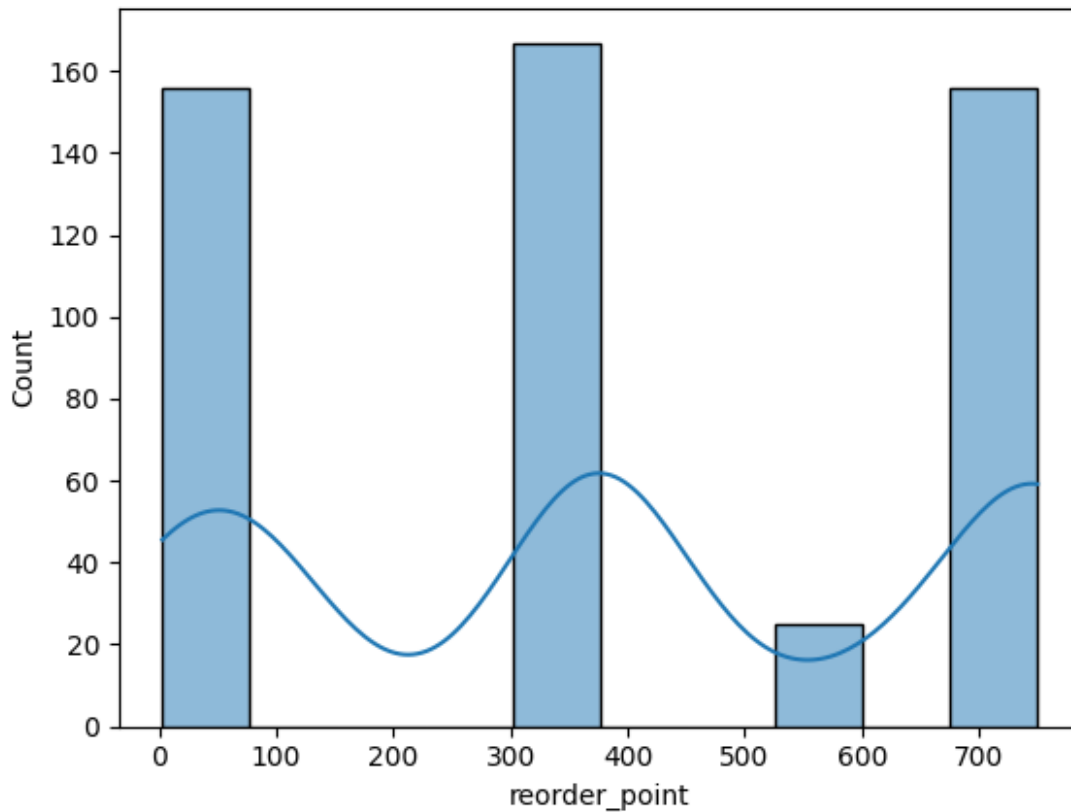
```
# Do not modify this code
print_tile(size="h3", key='feature_3_insights',
value=feature_3_insights)
```

<IPython.core.display.HTML object>

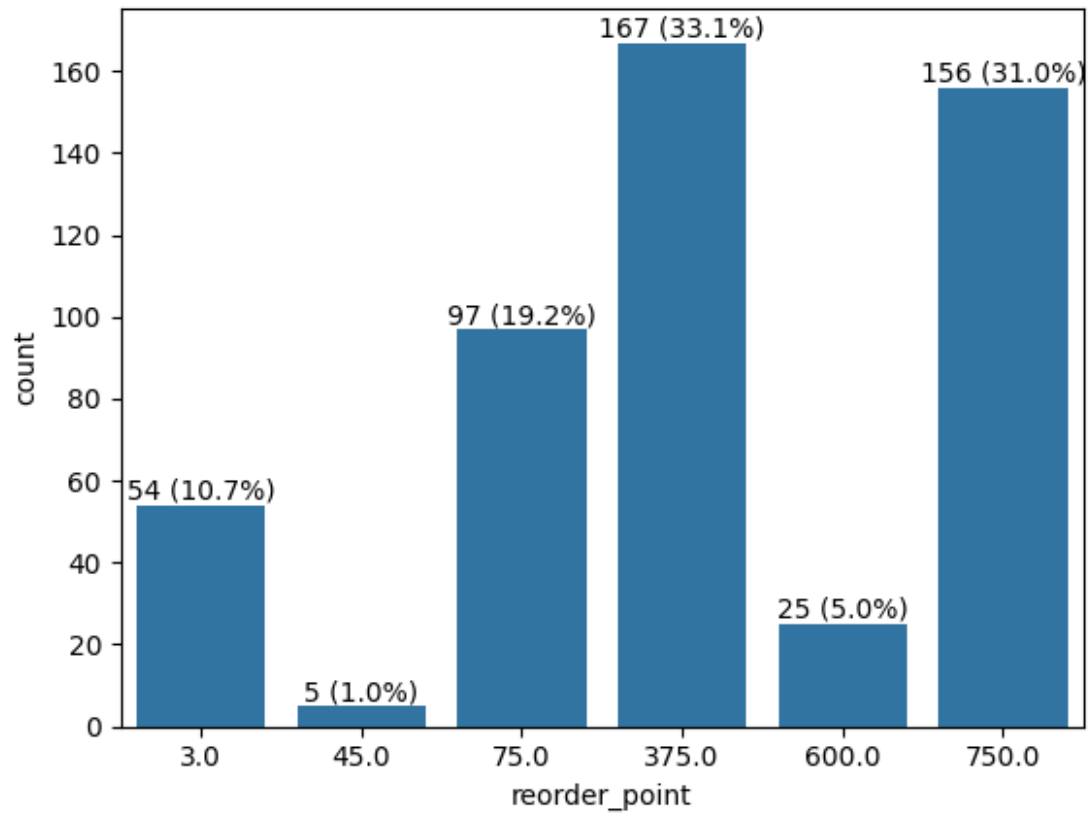
B.5 Explore Feature of Interest reorder_point

```
sns.histplot(df['reorder_point'], bins='auto', kde=True)
```

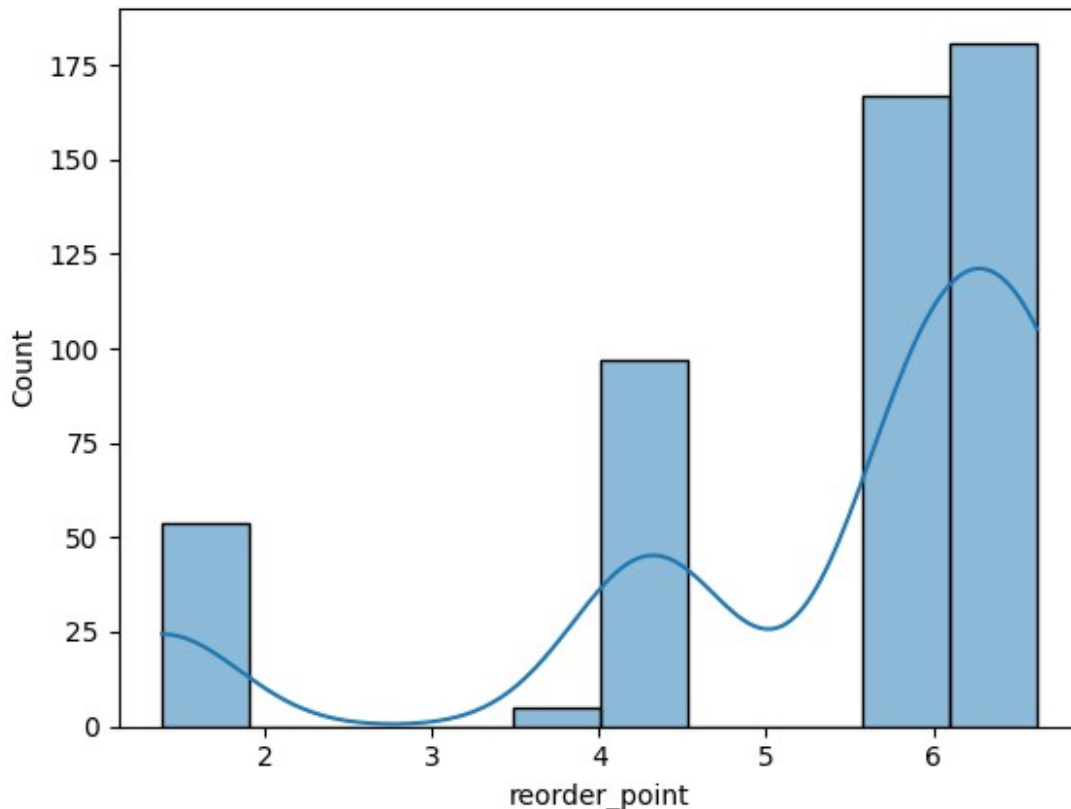
<Axes: xlabel='reorder_point', ylabel='Count'>



```
sns.barplot(x=df['reorder_point'].value_counts().index,
y=df['reorder_point'].value_counts())
total = len(df['reorder_point'])
for p in plt.gca().patches:
    percentage = '{:.1f}%'.format(100 * p.get_height()/total)
    x = p.get_x() + p.get_width() / 2
    y = p.get_height()
    plt.gca().annotate(f'{int(p.get_height())} ({percentage})', (x,
y), ha='center', va='bottom')
```



```
sns.histplot(np.log1p(df['reorder_point']), kde=True, bins='auto')  
<Axes: xlabel='reorder_point', ylabel='Count'>
```



```
df.groupby('reorder_point')['safety_stock_level'].value_counts()
```

```
reorder_point  safety_stock_level
3.0            4.0                  54
45.0           60.0                   5
75.0          100.0                  97
375.0         500.0                 167
600.0         800.0                  25
750.0        1000.0                 156
Name: count, dtype: int64
```

<Student to fill this section>

```
feature_4_insights = """
```

The 'reorder_point' feature represents the inventory level at which new stock needs to be restocked for a product. Similar to

'safety_stock_level', this

feature is numerically continuous but exhibits a limited number of distinct values (6 unique values), ranging from 3 to 750. Due to this a log

transformation could be beneficial for future analysis. A notable observation is the strong, consistent relationship between

'reorder_point' and

'safety_stock_level'. For every unique 'safety_stock_level', there is a corresponding unique 'reorder_point' value (e.g., a safety stock of

4.0
consistently has a reorder point of 3.0, and 1000.0 has 750.0). This suggests that 'reorder_point' is likely a derived value, calculated based on 'safety_stock_level' through a predefined business rule or formula, reflecting a structured inventory management policy.
"""

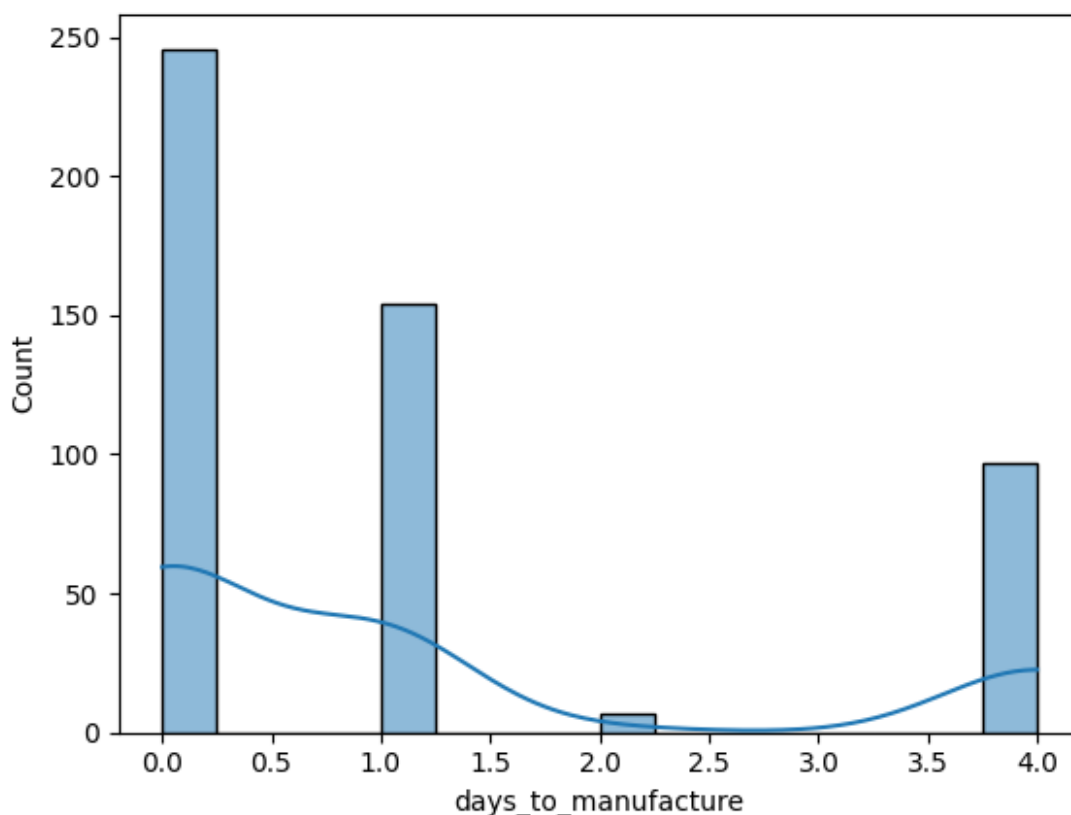
```
# Do not modify this code  
print_tile(size="h3", key='feature_4_insights',  
value=feature_4_insights)
```

<IPython.core.display.HTML object>

B.6 Explore Feature of Interest days_to_manufacture

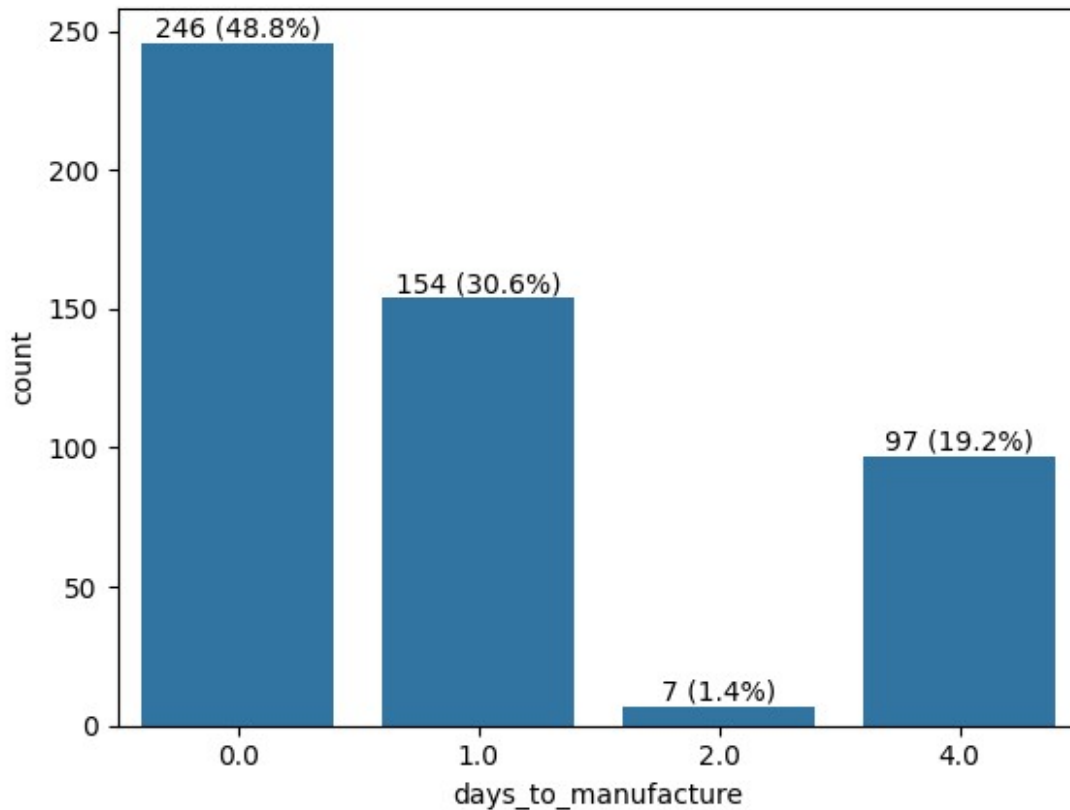
```
sns.histplot(df['days_to_manufacture'], bins='auto', kde=True)
```

<Axes: xlabel='days_to_manufacture', ylabel='Count'>



```
sns.barplot(x=df['days_to_manufacture'].value_counts().index,  
y=df['days_to_manufacture'].value_counts())  
total = len(df['days_to_manufacture'])  
for p in plt.gca().patches:
```

```
percentage = '{:.1f}%'.format(100 * p.get_height()/total)
x = p.get_x() + p.get_width() / 2
y = p.get_height()
plt.gca().annotate(f'{int(p.get_height())} ({percentage})', (x,
y), ha='center', va='bottom')
```



```
df.groupby('is_manufactured')['days_to_manufacture'].value_counts()
```

is_manufactured	days_to_manufacture	count
0.0	0.0	246
0.0	1.0	19
1.0	1.0	135
1.0	4.0	97
1.0	2.0	7

Name: count, dtype: int64

<Student to fill this section>

feature_5_insights = """

The 'days_to_manufacture' feature represents the estimated number of days required to produce or assemble a product. This feature is numerical but exhibits a very limited number of distinct integer values (0, 1, 2, 4). The distribution is heavily imbalanced, with 0 days being the most frequent, followed by 1,

```

4, and 2 days, respectively, with the lowest being 2, only containing
7 samples. When cross-referenced with the 'is_manufactured' feature -
Products not
manufactured, overwhelmingly have 'days_to_manufacture' as 0 (246
products), with a small exception of 19 products taking 1 day. This is
logical, as
products not manufactured in-house would not require manufacturing
time. Products manufactured in-house show varying
'days_to_manufacture' values: 1 day
(135 products), 4 days (97 products), and 2 days (7 products). This
distribution is expected for products that undergo an internal
manufacturing process,
reflecting different production complexities or lead times.
"""

```

```

# Do not modify this code
print_tile(size="h3", key='feature_5_insights',
value=feature_5_insights)

```

<IPython.core.display.HTML object>

B.7 Explore Feature of Interest product_subcategory_id

```

for csv_file in os.listdir(at.folder_path):
    if csv_file.endswith('.csv'):
        try:
            if 'product_subcategory_id' in pd.read_csv(at.folder_path
/ csv_file).columns:
                print(csv_file)
            except UnicodeDecodeError:
                print(f"Warning: Could not read {csv_file} with default
encoding. Skipping or consider specifying encoding.")
            except Exception as e:
                print(f"Error reading {csv_file}: {e}")

```

```

product_sub_category.csv
product.csv

```

```

df[df['product_subcategory_id'].isna()].groupby('is_manufactured')
['is_sellable'].value_counts()

```

```

is_manufactured  is_sellable
0.0              0.0          182
1.0              0.0           27
Name: count, dtype: int64

```

```

bool((df['is_sellable'] == 0).sum() ==
(df['product_subcategory_id'].isna().sum()))

```

True

```

# <Student to fill this section>
feature_6_insights = """
The 'product_subcategory_id' feature serves as a categorical
identifier for product subcategories. This feature contains 295 non-
null values. Notably, all
209 products that lack a `product_subcategory_id` are precisely those
products where `is_sellable` is 0 (i.e., 'Purchased & Not Sold' or
'Manufactured &
Not Sold'). This indicates a systematic pattern where subcategory IDs
are exclusively assigned to sellable products. This feature acts as a
foreign key,
linking the 'product' dataset to 'product_sub_category.csv', enabling
detailed analysis of product subcategories for sellable items.
"""

# Do not modify this code
print_tile(size="h3", key='feature_6_insights',
value=feature_6_insights)

<IPython.core.display.HTML object>

```

B.8 Explore Feature of Interest product_model_id

```

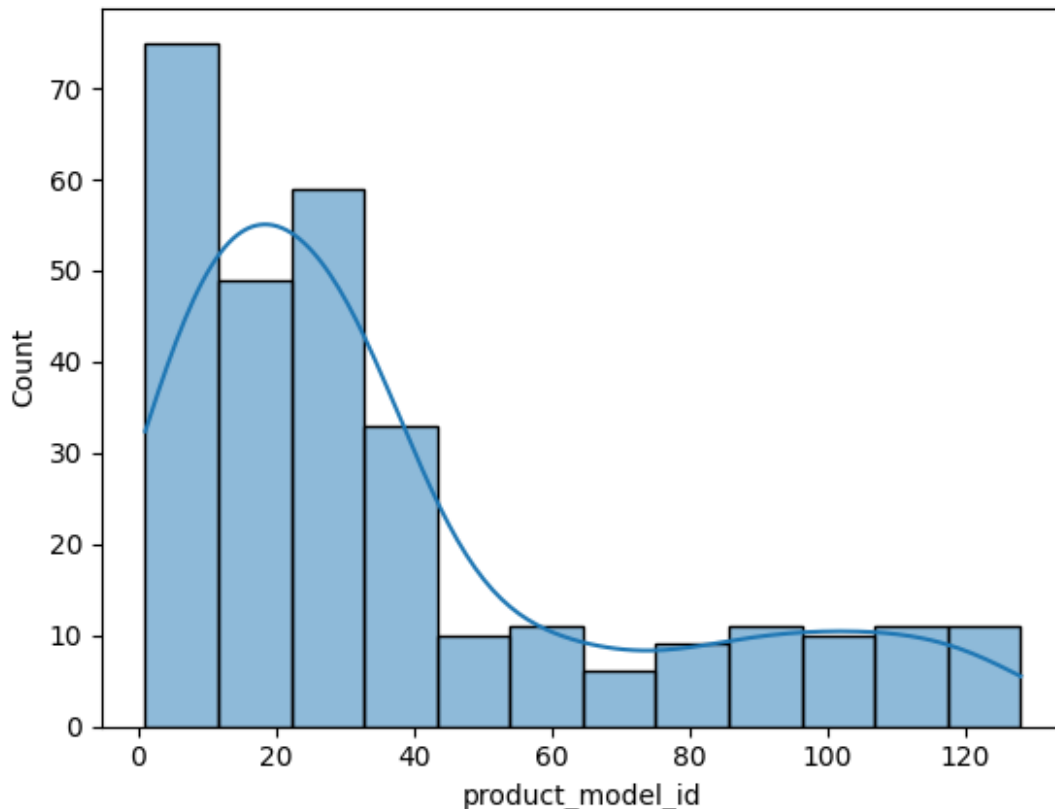
for csv_file in os.listdir(at.folder_path):
    if csv_file.endswith('.csv'):
        try:
            if 'product_model_id' in pd.read_csv(at.folder_path /
csv_file).columns:
                print(csv_file)
            except UnicodeDecodeError:
                print(f"Warning: Could not read {csv_file} with default
encoding. Skipping or consider specifying encoding.")
            except Exception as e:
                print(f"Error reading {csv_file}: {e}")

product.csv

sns.histplot(df['product_model_id'], bins='auto', kde=True)

<Axes: xlabel='product_model_id', ylabel='Count'>

```



```
df[df['product_model_id'].isna()].groupby('is_manufactured')
['is_sellable'].value_counts()
```

```
is_manufactured  is_sellable
0.0              0.0          182
1.0              0.0          27
Name: count, dtype: int64
```

```
bool((df['is_sellable'] == 0).sum() ==
(df['product_model_id'].isna().sum()))
```

True

<Student to fill this section>

```
feature_7_insights = """
```

The 'product_model_id' represents a 'Base Model' or 'Product Family', with small integer values (1 to 128) suggesting 128 distinct core product designs.

Similar to 'product_subcategory_id', all products with a missing 'product_model_id' are precisely those where 'is_sellable' is 0 (i.e., non-sellable items

like 'Purchased & Not Sold' or 'Manufactured & Not Sold'). This systematic pattern implies that 'product_model_id' is only assigned to sellable products, indicating that non-sellable components or raw materials do not have a

```
defined product model in the same way as products offered for sale.
Without an
external lookup table, this numerical ID primarily serves to group
related sellable products by design or family.
"""
```

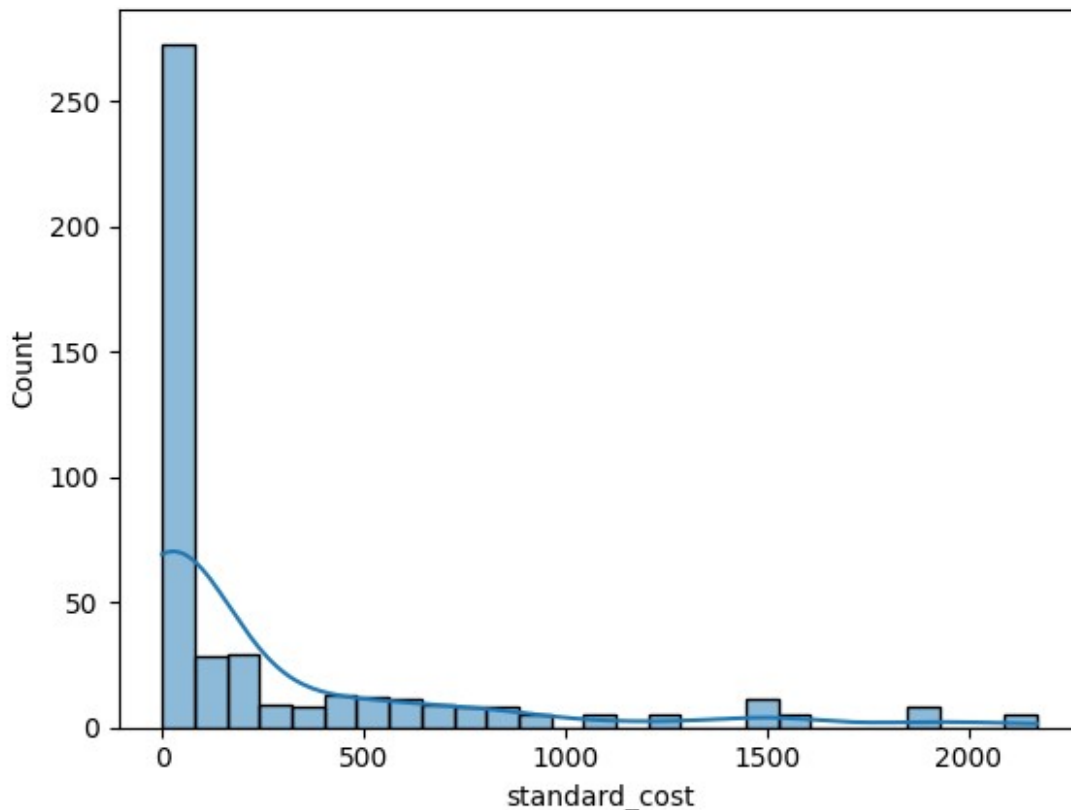
```
# Do not modify this code
print_tile(size="h3", key='feature_7_insights',
value=feature_7_insights)
```

<IPython.core.display.HTML object>

B.9 Explore Feature of Interest `standard_cost`

```
sns.histplot(df['standard_cost'], bins='auto', kde=True)
```

<Axes: xlabel='standard_cost', ylabel='Count'>



```
df['standard_cost'].value_counts().iloc[:30].sort_index()
```

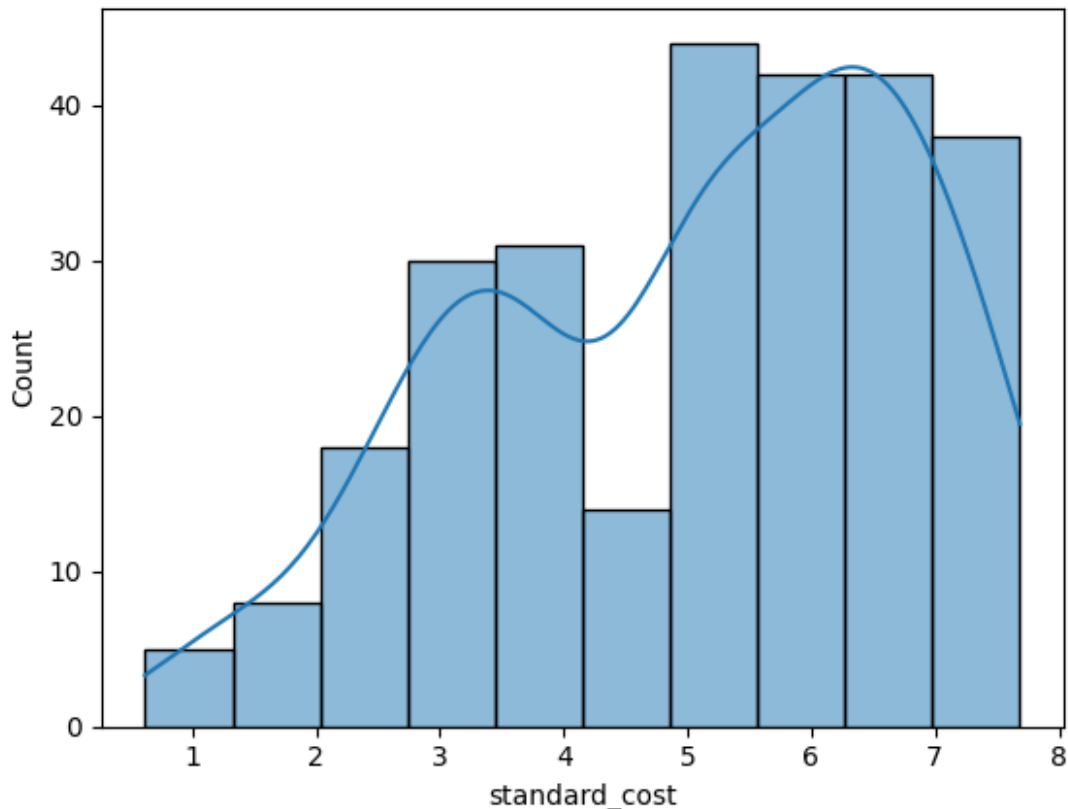
```
standard_cost
0.0000      180
24.7459       4
41.5723       4
```

98.7700	3
108.9900	3
136.7850	5
144.5938	5
185.8193	4
187.1571	6
199.3757	4
199.8519	8
204.6251	6
294.5797	4
308.2179	5
343.6496	3
461.4448	10
486.7066	12
598.4354	4
601.7437	7
713.0798	5
755.1508	4
868.6342	8
884.7083	5
1082.5100	4
1481.9379	8
1518.7864	3
1554.9479	5
1898.0944	4
1912.1544	4
2171.2942	5

Name: count, dtype: int64

```
sns.histplot(x=np.log1p(df[df['standard_cost'] > 0]['standard_cost']),  
bins='auto', kde=True)
```

```
<Axes: xlabel='standard_cost', ylabel='Count'>
```



```
df[df['standard_cost'] == 0].groupby('is_manufactured')
['is_sellable'].value_counts()
```

```
is_manufactured  is_sellable
0.0              0.0          164
1.0              0.0          16
Name: count, dtype: int64
```

```
df[(df['is_sellable'] == 0) & (df['standard_cost'] > 0)][['name',
'is_manufactured']]
```

```
{"summary":{"\n  \"name\": \"df[(df['is_sellable'] == 0) & (df['standard_cost'] > 0)][['name', 'is_manufactured']]\", \n  \"rows\": 8, \n  \"fields\": [\n    {\n      \"column\": \"name\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 7, \n        \"samples\": [\n          \"ML Touring Seat Assembly\", \n          \"ML Road Seat Assembly\", \n          \"HL Touring Seat Assembly\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"is_manufactured\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.0, \n          \"min\": 1.0, \n          \"max\": 1.0, \n          \"num_unique_values\": 1, \n          \"samples\": [\n            1.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        } \n      } \n    ] \n  }, \"type\": \"dataframe\"}
```

```
df[df['name'].str.contains('Seat Assembly', na=False)][['name',
'is_manufactured']]
```

```
{"summary":{"\n  \"name\": \"df[df['name']]\",\n  \"rows\": 7,\n  \"fields\": [\n    {\n      \"column\": \"name\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 7,\n        \"samples\": [\n          \"ML Touring Seat Assembly\", \n          \"ML Road Seat Assembly\", \n          \"HL Touring Seat Assembly\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"is_manufactured\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.0, \n          \"min\": 1.0, \n          \"max\": 1.0, \n          \"num_unique_values\": 1, \n          \"samples\": [\n            1.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        } \n      } \n    ] \n  }, \"type\": \"dataframe\"}
```

```
df[df['standard_cost'].isna()].groupby('is_manufactured')
['is_sellable'].value_counts()
```

is_manufactured	is_sellable	
0.0	0.0	18
	1.0	10
1.0	1.0	21
	0.0	3

Name: count, dtype: int64

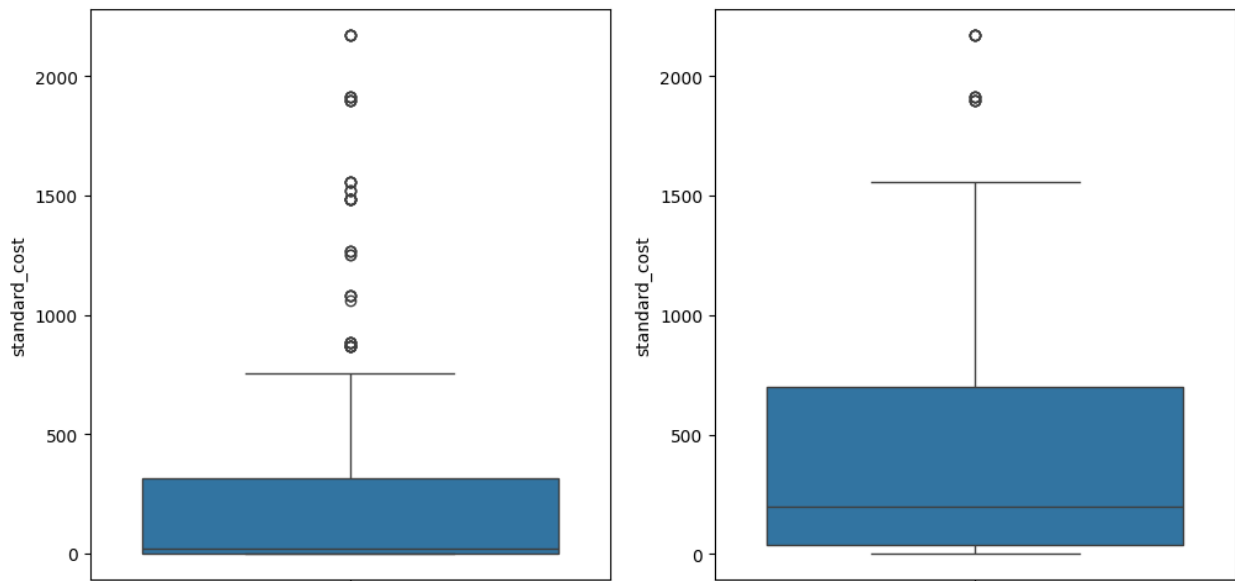
```
df[df['standard_cost'].isna()][['name', 'product_model_id',
'days_to_manufacture', 'reorder_point']]
```

```
{"summary":{"\n  \"name\": \"df[df['standard_cost']]\",\n  \"rows\": 52,\n  \"fields\": [\n    {\n      \"column\": \"name\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 37,\n        \"samples\": [\n          \"ML Mountain Rim\", \n          \"HL Touring Frame - Yellow, 54\", \n          \"Fender Set - Mountain\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"product_model_id\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 38.955281779171145, \n          \"min\": 1.0, \n          \"max\": 121.0, \n          \"num_unique_values\": 25, \n          \"samples\": [\n            4.0, \n            31.0, \n            5.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n      {\n        \"column\": \"days_to_manufacture\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 1.0823902380615693, \n          \"min\": 0.0, \n          \"max\": 4.0, \n          \"num_unique_values\": 4, \n          \"samples\": [\n            0.0, \n            4.0, \n            2.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        } \n      }, \n      {\n        \"column\": \"reorder_point\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 238.190719811929, \n          \"min\": 3.0, \n          \"max\": 750.0, \n          \"num_unique_values\": 1, \n          \"samples\": [\n            3.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        } \n      } \n    ] \n  }, \"type\": \"dataframe\"}
```

```
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n          750.0,\n          600.0,\n          3.0\n          ],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n          }\n          }\n          }\n          }", "type": "dataframe"}
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
sns.boxplot(y=df['standard_cost'], ax=axes[0])
sns.boxplot(y=df[df['standard_cost'] > 0]['standard_cost'],
ax=axes[1])
```

<Axes: ylabel='standard_cost'>



```
feature_8_insights = """
```

The 'standard_cost' feature represents the cost to produce or acquire a product. A significant portion of products (180 out of 504) have a 'standard_cost' of 0. Upon cross-referencing with 'is_manufactured' and 'is_sellable', it is observed that all products with a 'standard_cost' of 0 are not sellable.

Specifically, 164 of these products are 'Purchased & Not Sold' (likely raw materials or components) and 16 are 'Manufactured & Not Sold' (likely internal sub-assemblies). This systematic pattern indicates that a 'standard_cost' of 0 is not a data error, but rather a deliberate accounting or inventory management practice. It suggests that for non-sellable items, such as raw materials or intermediate components used in the manufacturing process, a direct individual standard cost is either not tracked separately or is incorporated into the cost structure of the final sellable products they support.

Additionally, products with 'is_sellable' equal to 0 but a non-zero

```
'standard_cost' are predominantly manufactured assembly-related components, specifically seat assemblies. Their non-zero cost indicates that these items participate in internal manufacturing and inventory accounting processes, even though they are not classified as final customer-facing sellable products. In contrast, the missing values observed in related product identification fields (such as 'product_number') do not exhibit a consistent structural or business-related pattern. The missing entries appear across different product categories, manufacturing statuses, and sellability groups, suggesting that these missing values are likely the result of incomplete data entry, migration issues, or data collection inconsistencies rather than intentional business logic. The box plot for 'standard_cost' (including zero values) shows a high concentration at 0, while the box plot excluding zero values shows the distribution of actual costs, highlighting potential outliers and the overall spread of non-zero standard costs.
"""
```

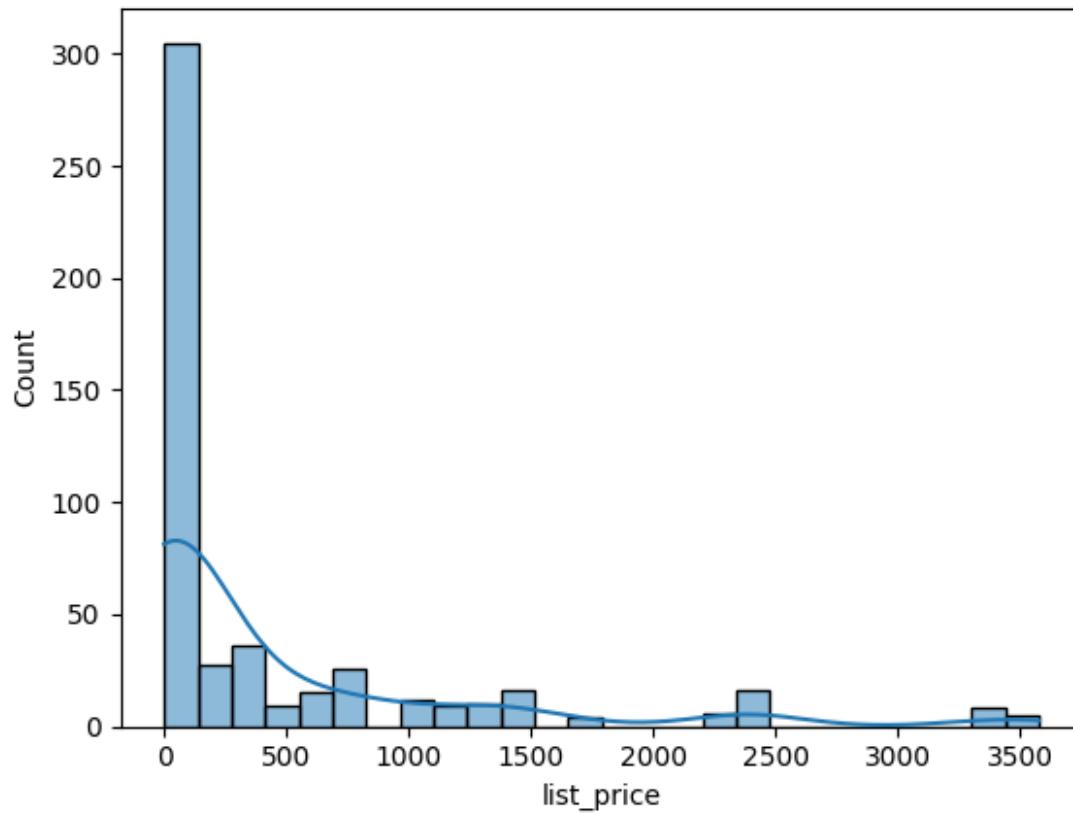
```
# Do not modify this code
print_tile(size="h3", key='feature_8_insights',
value=feature_8_insights)
```

```
<IPython.core.display.HTML object>
```

B.10 Explore Feature of Interest list_price

```
sns.histplot(df['list_price'], bins='auto', kde=True)
```

```
<Axes: xlabel='list_price', ylabel='Count'>
```



```
df['list_price'].value_counts().iloc[:30].sort_index()
```

```
list_price
0.00      200
8.99       4
34.99      4
53.99      5
59.99      4
249.79     5
264.05     5
333.42    10
337.22    12
348.76     4
364.09     4
539.99     9
564.99     5
594.83    10
742.35    10
769.49     4
782.99    12
1003.91     8
1079.99     4
1120.49     5
1214.85     4
```

```

1349.60      5
1364.50      5
1431.50     11
1457.99      5
1700.99      4
2384.07      8
2443.35      8
3374.99      4
3578.27      5
Name: count, dtype: int64

```

```

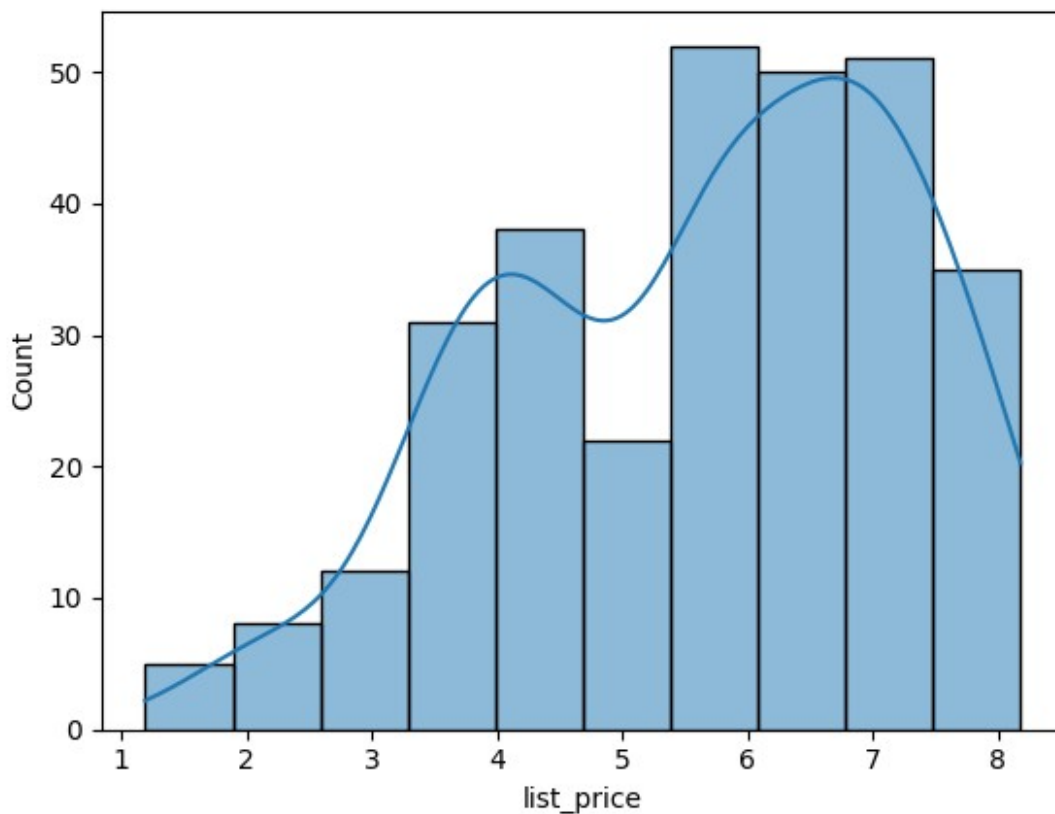
sns.histplot(x=np.log1p(df[df['list_price'] > 0]['list_price']),
bins='auto', kde=True)

```

```

<Axes: xlabel='list_price', ylabel='Count'>

```



```

df[df['list_price'] == 0].groupby('is_manufactured')
['is_sellable'].value_counts()

```

```

is_manufactured  is_sellable
0.0              0.0         182
1.0              0.0         18
Name: count, dtype: int64

```

```

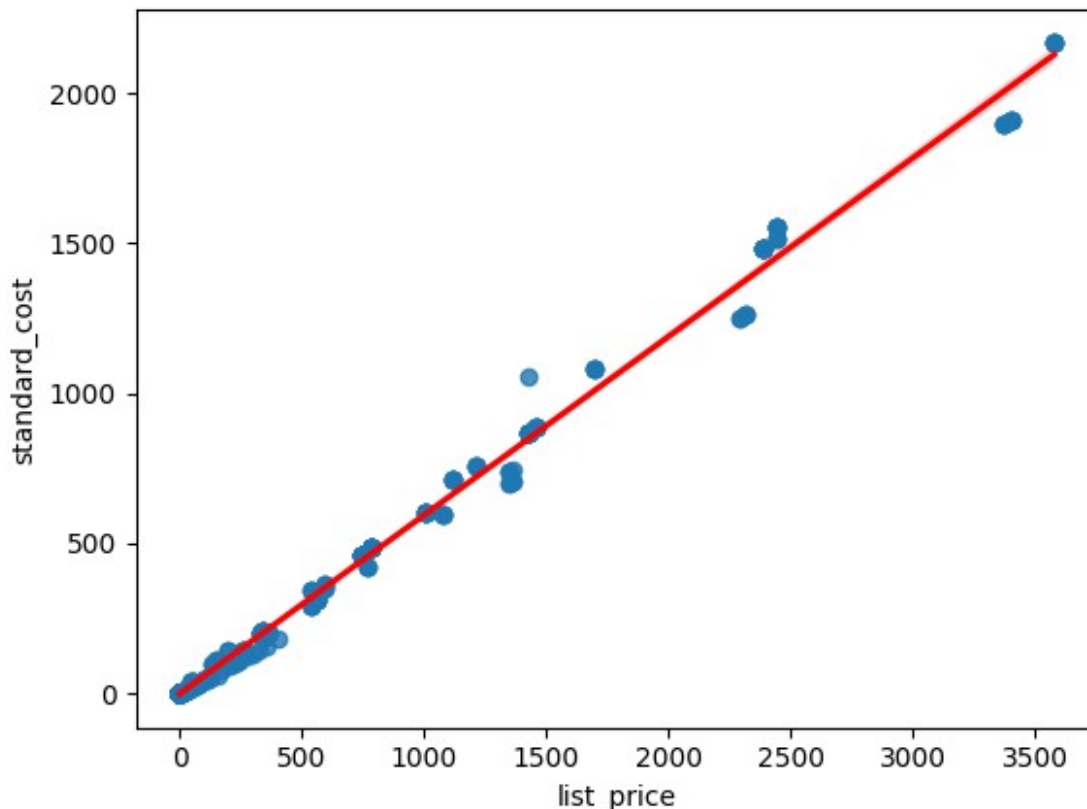
df[(df['is_sellable'] == 0) & (df['list_price'] > 0)][['name',
'is_manufactured']]

{"summary": "{\n  \"name\": \"df[(df['is_sellable'] == 0) & (df['list_price'] > 0)][['name', 'is_manufactured']]\", \n  \"rows\": 9, \n  \"fields\": [\n    {\n      \"column\": \"name\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 7, \n        \"samples\": [\n          \"ML Touring Seat Assembly\", \n          \"ML Road Seat Assembly\", \n          \"HL Touring Seat Assembly\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"is_manufactured\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.0, \n        \"min\": 1.0, \n        \"max\": 1.0, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1.0 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    ] \n  }, \n  \"type\": \"dataframe\"}

sns.regplot(data=df, x='list_price', y='standard_cost',
line_kws={'color': 'red'})

<Axes: xlabel='list_price', ylabel='standard_cost'>

```



```
df[df['standard_cost'] == 0]['list_price'].sum()
```

```

np.float64(0.0)

len(df['standard_cost'].unique()), len(df['list_price'].unique())

(109, 103)

feature_9_insights = """
The 'list_price' feature represents the price at which a product is
sold to customers. Its distribution, as shown by the histogram,
indicates a right-
skewed pattern, with many products having lower list prices and fewer
products at higher price points. A log transformation of list prices
could help
normalize this distribution for better analysis in future. A
significant observation is the presence of 200 products with a
'list_price' of 0. Upon cross-
referencing with 'is_manufactured' and 'is_sellable', it's evident
that these products align systematically with non-sellable items - 182
products are
'Purchased & Not Sold' and 18 products are 'Manufactured & Not Sold'.
This pattern suggests that a 'list_price' of 0 is not an error but a
deliberate
reflection of products not intended for direct sale to customers, such
as raw materials, components, or internal sub-assemblies. Their value
is likely
incorporated into the list price of final, sellable products.
Conversely, a small number of products (9 entries) have 'is_sellable'
equal to 0 but a
'list_price' greater than 0. These products are primarily 'Seat
Assembly' components, which are manufactured in-house. This indicates
that while they
might not be classified as sellable as a finished good in the primary
product catalog, they might be available as spare parts or replacement
items, hence
retaining a non-zero list price. Furthermore, there is a strong linear
relationship between 'list_price' and 'standard_cost', as depicted by
the scatter
plot and regression line. This linear correlation suggests that if
imputation for 'standard_cost' is required, the 'list_price' feature
could serve as a
valuable predictor. All products with a 'list_price' of 0 consistently
have a 'standard_cost' of 0, indicating a clear, direct relationship
for these
specific cases. However, since the number of unique values for
'standard_cost' and 'list_price' are not identical, a direct one-to-
one mapping cannot be
assumed for all values, reinforcing the utility of a regression-based
imputation approach rather than a simple direct mapping.
"""

```

```
# Do not modify this code
print_tile(size="h3", key='feature_9_insights',
value=feature_9_insights)
```

<IPython.core.display.HTML object>

B.11 Explore Feature of Interest product_number

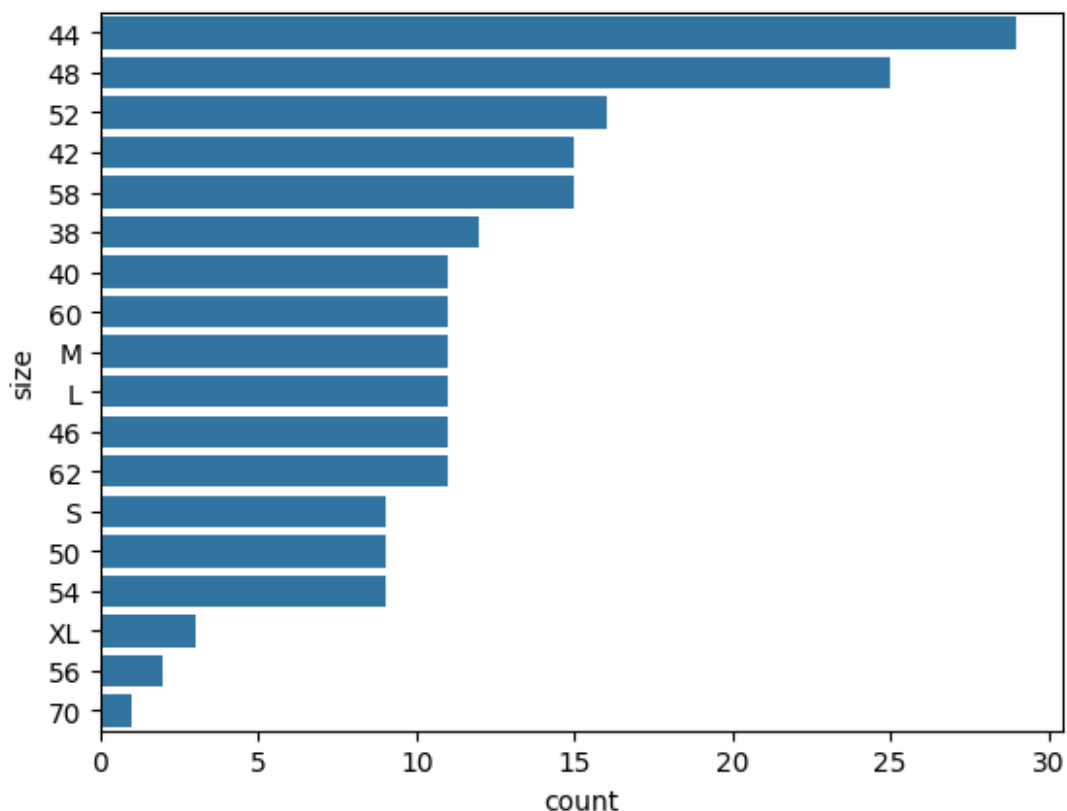
```
df[df['product_number'].isna()].groupby('is_sellable')
['is_manufactured'].value_counts()
```

is_sellable	is_manufactured	
0.0	0.0	15
	1.0	3
1.0	1.0	19
	0.0	5

Name: count, dtype: int64

```
df[(df['product_number'].isna())][['name', 'product_model_id',
'days_to_manufacture', 'reorder_point', 'list_price']].sample(25)
```

```
{
  "summary": {
    "\n  \"name\": \"df[(df['product_number'].isna())][['name', 'product_model_id',\n  \"rows\": 25,\n  \"fields\": [\n    {\n      \"column\": \"name\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 15, \n        \"samples\": [\n          \"LL Road Frame - Black, 48\", \n          \"ML Mountain Rear Wheel\", \n          \"Mountain-300 Black, 38\", \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      }, \n    }, \n    {\n      \"column\": \"product_model_id\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 36.41912108978443, \n        \"min\": 6.0, \n        \"max\": 124.0, \n        \"num_unique_values\": 13, \n        \"samples\": [\n          19.0, \n          29.0, \n          21.0, \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      }, \n    }, \n    {\n      \"column\": \"days_to_manufacture\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 1.6852299546352714, \n        \"min\": 0.0, \n        \"max\": 4.0, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          4.0, \n          0.0, \n          1.0, \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      }, \n    }, \n    {\n      \"column\": \"reorder_point\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 289.3104560848087, \n        \"min\": 3.0, \n        \"max\": 750.0, \n        \"num_unique_values\": 6, \n        \"samples\": [\n          75.0, \n          750.0, \n          45.0, \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n      }, \n    }, \n    {\n      \"column\": \"list_price\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 914.3687536245063, \n        \"min\": 0.0, \n        \"max\": 3374.99, \n        \"num_unique_values\": 14, \n        \"samples\": [\n          1214.85, \n          62.09, \n        ], \n      }, \n    } \n  ] \n}
```

```
df[(df['name'].notna()) & (df['size'].notna())]['name'].sample(25)
```

```
324    ML Mountain Frame-W - Silver, 38
880      LL Mountain Frame - Black, 44
226          Road-150 Red, 44
825      LL Mountain Frame - Silver, 40
41          Road-550-W Yellow, 42
434      HL Road Frame - Red, 52
792      Mountain Bike Socks, L
377      Mountain-500 Silver, 52
869    Short-Sleeve Classic Jersey, M
876      Mountain-100 Silver, 38
29      HL Touring Frame - Yellow, 50
234      ML Mountain Frame - Black, 40
89          Road-650 Red, 48
516      Mountain-300 Black, 48
400      Half-Finger Gloves, M
432      Road-650 Red, 52
51      LL Touring Frame - Blue, 58
285      Mountain-100 Black, 42
658      HL Mountain Frame - Black, 38
189      LL Touring Frame - Yellow, 50
17      Short-Sleeve Classic Jersey, XL
104      Full-Finger Gloves, L
```

```

156          Mountain-200 Silver, 42
155      HL Mountain Frame - Silver, 44
347          Women's Tights, S
Name: name, dtype: object

```

```
df[(df['name'].notna()) & (df['size'].isna())]['name'].sample(25)
```

```

709      LL Touring Seat/Saddle
597      HL Touring Seat Assembly
98          Road Tire Tube
134          Thin-Jam Lock Nut 7
244          Internal Lock Washer 8
61          HL Mountain Rear Wheel
112          Flat Washer 4
867          Thin-Jam Hex Nut 11
74          Fender Set - Mountain
100          Lock Nut 18
676          Thin-Jam Lock Nut 3
394          Patch Kit/8 Patches
328          Reflector
581          Flat Washer 7
685          Headset Ball Bearings
487          LL Road Front Wheel
71          Hex Nut 12
596          ML Road Seat/Saddle
375          Hex Nut 14
725          HL Headset
603          Lock Nut 17
210          Chainring Bolts
24          HL Fork
399          Mountain Tire Tube
856      LL Mountain Seat Assembly
Name: name, dtype: object

```

```
df[df['size'].isna()].groupby('is_manufactured')
['is_sellable'].value_counts()
```

is_manufactured	is_sellable	
0.0	0.0	182
	1.0	48
1.0	1.0	36
	0.0	27

```
Name: count, dtype: int64
```

```
df.groupby('is_manufactured')['is_sellable'].value_counts()
```

is_manufactured	is_sellable	
0.0	0.0	182
	1.0	83
1.0	1.0	212

Name: count, dtype: int64

```
df[(df['size'].isna()) & (df['is_sellable'] ==
1)].describe(include='all').T
```

```
{
  "summary": {
    "name": "df[(df['size'].isna()) & (df['is_sellable'] == 1)].describe(include='all').T",
    "fields": [
      {
        "column": "count",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 84,
          "num_unique_values": 11,
          "samples": [0, 84, 1]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "unique",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 84,
          "num_unique_values": 9,
          "samples": [4, 23, 0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "top",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 84,
          "num_unique_values": 11,
          "samples": [
            "8c4f020f-f9c1-492b-8395-1372b7bda7a7",
            "71ceelfa-a7d3-423a-8bf8-d586f864dc30",
            "2013-05-30 00:00:00"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "freq",
        "properties": {
          "dtype": "date",
          "min": 1,
          "max": 47,
          "num_unique_values": 7,
          "samples": [1, 14, 47]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "mean",
        "properties": {
          "dtype": "date",
          "min": 0.42857142857142855,
          "max": 444.1818181818182,
          "num_unique_values": 9,
          "samples": [93.01642857142856, 1.0, 82.71428571428571]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "std",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 322.1080322121515,
          "num_unique_values": 9,
          "samples": [92.98685861800816, 0.0, 28.651445611491006]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "min",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 88.0,
          "num_unique_values": 8,
          "samples": [1.0, 0.8565, 0.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "25%",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [12.0413, 0.0]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "50%",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [12.0413, 0.0]
        },
        "semantic_type": "",
        "description": ""
      }
    ]
  }
}
```

```

{"date": 23.3722, "min": 0.0, "max": 500.0, "num_unique_values": 8, "samples": [1.0, 0.0], "semantic_type": "date", "description": "75%", "column": "75%", "properties": {"dtype": "date", "min": 1.0, "max": 646.25, "num_unique_values": 7, "samples": [1.0, 500.0, 120.5675]}, "semantic_type": "date", "description": "max", "column": "max", "properties": {"dtype": "date", "min": 1.0, "max": 1050.0, "num_unique_values": 7, "samples": [1.0, 500.0, 404.99]}, "semantic_type": "date", "description": ""}, {"type": "dataframe"}

```

```

temp = df[(df['size'].isin(['XL', 'M', 'L', 'S'])) &
(df['name'].notna())]['name']
print(len(temp))
display(temp)

```

28

```

17    Short-Sleeve Classic Jersey, XL
43    Men's Sports Shorts, L
80    Long-Sleeve Logo Jersey, M
104   Full-Finger Gloves, L
110   Classic Vest, S
137   Half-Finger Gloves, L
168   Mountain Bike Socks, M
225   Men's Sports Shorts, M
263   Long-Sleeve Logo Jersey, S
281   Short-Sleeve Classic Jersey, S
296   Classic Vest, L
331   Racing Socks, L
347   Women's Tights, S
359   Full-Finger Gloves, M
400   Half-Finger Gloves, M
419   Women's Tights, M
448   Classic Vest, M
480   Long-Sleeve Logo Jersey, L
544   Women's Mountain Shorts, S
548   Men's Sports Shorts, S
572   Full-Finger Gloves, S
693   Men's Bib-Shorts, L
744   Long-Sleeve Logo Jersey, XL
761   Women's Mountain Shorts, L
786   Men's Sports Shorts, XL
792   Mountain Bike Socks, L
806   Men's Bib-Shorts, M

```

869 Short-Sleeve Classic Jersey, M

Name: name, dtype: object

```
clothing_keywords = ['jersey', 'shorts', 'socks', 'gloves', 'vest',
'tights']
clothing_df = df[df['name'].str.contains('|'.join(clothing_keywords),
case=False, na=False)]
print(len(clothing_df))
```

28

```
temp = df[(df['size_unit_measure_code'].isna()) & (df['size'].notna())
& (df['name'].notna())][['name', 'size']]
print(len(temp))
display(temp)
```

29

```
{"summary":{"\n  \"name\": \"temp\",\n  \"rows\": 29,\n  \"fields\": [\n    {\n      \"column\": \"name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 29,\n        \"samples\": [\n          \"Men's Bib-Shorts, M\",\n          \"Classic Vest, M\",\n          \"Women's Tights, S\"],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"size\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"L\",\n          \"70\",\n          \"M\"],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"temp\"}
```

```
for csv_file in os.listdir(at.folder_path):
    if csv_file.endswith('.csv'):
        try:
            if 'size_unit_measure_code' in pd.read_csv(at.folder_path
/ csv_file).columns:
                print(csv_file)
            except UnicodeDecodeError:
                print(f"Warning: Could not read {csv_file} with default
encoding. Skipping or consider specifying encoding.")
            except Exception as e:
                print(f"Error reading {csv_file}: {e}")
```

product.csv

os.listdir(at.folder_path)

```
['customer.csv',
'person.csv',
'product_category.csv',
'product_cost_history.csv',
'product_list_price_history.csv',
```

```

'product_sub_category.csv',
'product.csv',
'sales_order_detail.csv',
'sales_order_header.csv',
'sales_territory.csv',
'special_offer_product.csv',
'special_offer.csv',
'store.csv',
'unit_measure.csv',
'scaler.pkl',
'X_train.csv']

unit_measure_df = pd.read_csv(at.folder_path / 'unit_measure.csv')
unique_codes =
pd.Series(df['size_unit_measure_code'].dropna().unique())
unit_measure_df[

unit_measure_df['unit_measure_code'].isin(unique_codes.sample(n=min(5,
len(unique_codes)), replace=False))]

{"summary":{"\n  \"name\": \"
unit_measure_df['unit_measure_code']\", \"rows\": 1, \"fields\":
[\n    {\n      \"column\": \"unit_measure_code\", \"
\"properties\": {\n        \"dtype\": \"string\", \"
\"num_unique_values\": 1, \"samples\": [\n
\"e2e52698-ce31-4fb2-866f-bcffbdfd3147\", \"
\"semantic_type\": \"\", \"description\": \"\"\"\"
n    }, \"column\": \"name\", \"properties\": {\n
\"dtype\": \"string\", \"num_unique_values\": 1, \"
\"samples\": [\n      \"Centimeter\", \"
\"semantic_type\": \"\", \"description\": \"\"\"\"
n    }]\n} \", \"type\": \"dataframe\"}

feature_11_insights = ""
The 'size' feature, despite being stored as an object type, contains a
mix of numerical and categorical values, indicating varying sizing
conventions across
products. Non-numeric size values such as 'XL', 'M', 'L' and 'S' are
associated with clothing products. A significant portion of products
have missing 'size'
values (293 out of 504 products). Upon inspection, these products are
primarily accessories, components, or non-essential items like 'Lock
Nut', 'Washer',
'Decal', 'Saddle', 'Fender Set', 'Steerer', 'Tension Pulley', 'Hitch
Rack', and 'Bike Wash'. Furthermore, most of these products are often
associated with
'is_sellable == 0' (non-sellable) or are less critical components.
This suggests that sizing information is either irrelevant or not
tracked for these types
of products, which are typically not the primary revenue-generating

```

items. Conversely, essential products such as bicycle frames ('Mountain Frame', 'Road Frame'), complete bikes ('Mountain-100', 'Road-250', 'Touring-3000'), and other key components often have well-defined numeric values. This indicates that 'size' is a critical attribute for primary products, impacting their fit, compatibility, and market segmentation. The 'size_unit_measure_code' feature complements 'size' by providing the unit of measurement. When present, this code consistently maps to 'Centimeter', indicating that numerical sizes are recorded in centimeters. Notably, 'size_unit_measure_code' is missing (NaN) for all clothing products (that have categorical sizes like 'XL', 'M', 'L', and 'S'), suggesting that these non-numeric size values do not have an explicit unit within the dataset. An interesting exception is 'Hydration Pack - 70 oz.', which has a numerical size of '70' but is missing a 'size_unit_measure_code'.

```
# Do not modify this code
print_tile(size="h3", key='feature_11_insights',
value=feature_11_insights)
```

<IPython.core.display.HTML object>

B.13 Explore Feature of Interest weight and weight_unit_measure_code

```
df[df['weight'].isna()].describe(include='all').T
```

```
{
  "summary": {
    "name": "df[df['weight']]",
    "rows": 23,
    "fields": [
      {
        "column": "count",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 299,
          "num_unique_values": 11,
          "samples": [
            271.0,
            299
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "unique",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 299,
          "num_unique_values": 10,
          "samples": [
            3,
            27,
            5
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "top",
        "properties": {
          "dtype": "category",
          "num_unique_values": 11,
          "samples": [
            "M",
            "0ebd0784-173e-45b5-b534-a953030f49f0",
            "2008-04-30 00:00:00"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "freq",
        "properties": {
          "dtype": "date",
          "min": 1,
          "max": 202
        }
      }
    ]
  }
}
```



```
70      1
Name: count, dtype: int64
```

```
df[(df['size'].isna()) &
(df['weight'].notna())].describe(include='all').T
```

```
{
  "summary": {
    "name": "df[(df['size'].isna()) & (df['weight'].notna())].describe(include='all').T",
    "rows": 23,
    "fields": [
      {
        "column": "count",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 29,
          "num_unique_values": 8,
          "samples": [
            22,
            0,
            29
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "unique",
        "properties": {
          "dtype": "date",
          "min": 0,
          "max": 29,
          "num_unique_values": 8,
          "samples": [
            6,
            0,
            29
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "top",
        "properties": {
          "dtype": "category",
          "num_unique_values": 10,
          "samples": [
            "2013-05-30 00:00:00",
            "94ce21f6-249d-468d-a859-a7cac5f87bb0",
            "8c4f020f-f9c1-492b-8395-1372b7bda7a7"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "freq",
        "properties": {
          "dtype": "date",
          "min": 1,
          "max": 29,
          "num_unique_values": 6,
          "samples": [
            1,
            6,
            16
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "mean",
        "properties": {
          "dtype": "date",
          "min": 0.4827586206896552,
          "max": 572.4137931034483,
          "num_unique_values": 8,
          "samples": [
            0.7586206896551724,
            53.74008571428571,
            0.4827586206896552
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "std",
        "properties": {
          "dtype": "date",
          "min": 0.4354941703556927,
          "max": 279.21872881697516,
          "num_unique_values": 8,
          "samples": [
            0.4354941703556927,
            49.85101310521468,
            0.5085476277156077
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "min",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 500.0,
          "num_unique_values": 5,
          "samples": [
            500.0,
            88.0,
            375.0
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "25%",
        "properties": {
          "dtype": "date",
          "min": 0.0,
          "max": 500.0,
          "num_unique_values": 8,
          "samples": [
            1.0,
            23.9716,
            0.0
          ],
          "semantic_type": ""
        },
        "description": ""
      }
    ]
  }
}
```

```

\"50%\", \n      \"properties\": { \n          \"dtype\": \"date\", \n          \"min\": 0.0, \n          \"max\": 500.0, \n          \"num_unique_values\": 8, \n          \"samples\": [ \n              1.0, \n              40.6216, \n              0.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n      }, \n      { \n          \"column\": \"75%\", \n          \"properties\": { \n              \"dtype\": \"date\", \n              \"min\": 1.0, \n              \"max\": 600.0, \n              \"num_unique_values\": 7, \n              \"samples\": [ \n                  1.0, \n                  500.0, \n                  121.49 \n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\" \n          }, \n          { \n              \"column\": \"max\", \n              \"properties\": { \n                  \"dtype\": \"date\", \n                  \"min\": 1.0, \n                  \"max\": 1050.0, \n                  \"num_unique_values\": 7, \n                  \"samples\": [ \n                      1.0, \n                      800.0, \n                      404.99 \n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\" \n              } \n          } \n      ] \n  }, \"type\": \"dataframe\"}

```

```

unique_codes =
pd.Series(df['weight_unit_measure_code'].dropna().unique())
unit_measure_df[

```

```

unit_measure_df['unit_measure_code'].isin(unique_codes.sample(n=min(5,
len(unique_codes))), replace=False))]

```

```

{"summary": "{ \n      \"name\": \"unit_measure_df['unit_measure_code']\", \n      \"rows\": 2, \n      \"fields\": [ \n          { \n              \"column\": \"unit_measure_code\", \n              \"properties\": { \n                  \"dtype\": \"string\", \n                  \"num_unique_values\": 2, \n                  \"samples\": [ \n                      \"c4797df8-be0b-4d3a-a6c4-0f1765f0a822\", \n                      \"8c4f020f-f9c1-492b-8395-1372b7bda7a7\" \n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\" \n              }, \n              { \n                  \"column\": \"name\", \n                  \"properties\": { \n                      \"dtype\": \"string\", \n                      \"num_unique_values\": 2, \n                      \"samples\": [ \n                          \"US pound\", \n                          \"Gram\" \n                      ], \n                      \"semantic_type\": \"\", \n                      \"description\": \"\" \n                  } \n              } \n          ] \n      }, \"type\": \"dataframe\"}

```

```
feature_12_insights = ""
```

The 'weight' feature represents the mass of a product, while 'weight_unit_measure_code' indicates the unit of measurement used (either 'Gram' or 'US pound'). A significant portion of the products in the dataset have missing 'weight' values. Upon inspection, it's observed that the products with existing 'size' information, particularly categorical clothing sizes like 'M', 'L', 'S', and 'XL'. All the products with missing 'weight' values also don't have a 'size' value (expect clothing products), but the opposite is not true. There are products without any 'size' value but have a missing

'weight' value. This suggests that for apparel and potentially some other non-hardware items, weight might not be a tracked attribute, or it's considered less relevant. Conversely, for non-clothing products, the pattern of missing 'weight' data appears random, with some products having values and others not, suggesting a data entry or data collection inconsistency rather than a systematic business rule. The presence of two distinct weight units ('Gram' and 'US pound') indicates a need for unit standardization if weight is to be used in quantitative analysis or comparisons, to ensure consistency and avoid misinterpretations of product mass.

```

# Do not modify this code
print_tile(size="h3", key='feature_12_insights',
value=feature_12_insights)

```

<IPython.core.display.HTML object>

B.14 Explore Feature of Interest product_line

```
df['product_line'].value_counts()
```

```
product_line
```

```
R    100
```

```
M     91
```

```
T     52
```

```
S     35
```

```
Name: count, dtype: int64
```

```
df[df['product_line'].isna()].groupby('is_manufactured')
['is_sellable'].value_counts()
```

```
is_manufactured  is_sellable
```

```
0.0              0.0          182
```

```
              1.0           3
```

```
1.0              0.0          27
```

```
              1.0          14
```

```
Name: count, dtype: int64
```

```
display(df[(df['product_line'].isna()) & (df['is_sellable'] == 1) &
(df['is_manufactured'] == 0)]['name'])
```

```
display(df[(df['product_line'].isna()) & (df['is_sellable'] == 1) &
(df['is_manufactured'] == 1)]['name'])
```

```
365    Front Brakes
```

```
636    Rear Brakes
```

```
764           NaN
```

```
Name: name, dtype: object
```

```

24          HL Fork
153         LL Headset
162         ML Crankset
212          NaN
338         LL Crankset
416          NaN
446    Front Derailleur
498         HL Crankset
501         ML Headset
561          NaN
623         ML Fork
725         HL Headset
768    Rear Derailleur
796    LL Bottom Bracket
Name: name, dtype: object

```

```

for product_line in ['M', 'R', 'T', 'S']:
    print(product_line)
    print(list(df[(df['product_line'] == product_line) &
(df['name'].notna())]['name'].sample(5, random_state=42))))

```

M

```

['ML Mountain Seat/Saddle', 'Mountain-500 Black, 40', 'ML Mountain
Frame-W - Silver, 38', 'HL Mountain Front Wheel', 'Fender Set -
Mountain']

```

R

```

['HL Road Frame - Black, 48', 'Road-550-W Yellow, 44', 'Road-250 Red,
52', 'LL Road Pedal', 'ML Road Frame - Red, 52']

```

T

```

['Touring-2000 Blue, 46', 'HL Touring Frame - Blue, 60', 'LL Touring
Frame - Blue, 54', 'Touring-1000 Blue, 60', 'Touring-Panniers, Large']

```

S

```

['Water Bottle - 30 oz.', 'Half-Finger Gloves, M', 'Minipump',
'Classic Vest, M', 'Hitch Rack - 4-Bike']

```

```

df[df['product_line'].isna()].describe(include='all').T

```

```

{"summary":{"\n  \"name\": \"df[df['product_line']]\",\n  \"rows\":
23,\n  \"fields\": [\n    {\n      \"column\": \"count\", \n
\"properties\": {\n        \"dtype\": \"date\", \n        \"min\":
\"0\", \n        \"max\": \"226\", \n        \"num_unique_values\": 9, \n
\"samples\": [\n          \"30\", \n          \"17\", \n          201.0\
n        ], \n        \"semantic_type\": \"\", \n
\"description\": \"\" \n      }, \n      {\n        \"column\":
\"unique\", \n        \"properties\": {\n          \"dtype\": \"date\", \n
\"min\": 0, \n          \"max\": 226, \n          \"num_unique_values\": 8, \
n          \"samples\": [\n            7, \n            0, \n            226\
n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n
        }, \n        {\n          \"column\": \"top\", \n          \"properties\": {\
n          \"dtype\": \"category\", \n          \"num_unique_values\": 9, \n

```

```

\"samples\": [\n          \"2008-04-30 00:00:00\", \n          \"d1d7c22b-3508-4c1c-ae1a-1002f0bddc10\", \n          \"8c4f020f-f9c1-492b-8395-1372b7bda7a7\", \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"freq\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": \"1\", \n          \"max\": \"209\", \n          \"num_unique_values\": 7, \n          \"samples\": [\n          \"1\", \n          \"3\", \n          \"209\", \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"mean\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0.0752212389380531, \n          \"max\": 868.5840707964602, \n          \"num_unique_values\": 9, \n          \"samples\": [\n          16.8429203539823, \n          0.0752212389380531, \n          96.52941176470588, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"std\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0.2643334505090375, \n          \"max\": 226.22453843378062, \n          \"num_unique_values\": 9, \n          \"samples\": [\n          53.244645488644274, \n          0.2643334505090375, \n          19.774091784007503, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"min\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0.0, \n          \"max\": 88.0, \n          \"num_unique_values\": 5, \n          \"samples\": [\n          60.0, \n          88.0, \n          45.0, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"25%\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0.0, \n          \"max\": 800.0, \n          \"num_unique_values\": 5, \n          \"samples\": [\n          800.0, \n          223.0, \n          600.0, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"50%\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0.0, \n          \"max\": 1000.0, \n          \"num_unique_values\": 5, \n          \"samples\": [\n          1000.0, \n          400.0, \n          750.0, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"75%\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0.0, \n          \"max\": 1000.0, \n          \"num_unique_values\": 5, \n          \"samples\": [\n          1000.0, \n          450.0, \n          750.0, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          {\n          \"column\": \"max\", \n          \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 1.0, \n          \"max\": 1000.0, \n          \"num_unique_values\": 7, \n          \"samples\": [\n          1.0, \n          1000.0, \n          404.99, \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          }, \n          }\n          ], \n          \"type\": \"dataframe\"}

```

```
feature_13_insights = ""
```

The 'product_line' feature categorizes products into four distinct

```

lines: 'M' (Mountain bikes and accessories), 'R' (Road bikes and
accessories), 'T'
(Touring bikes and accessories), and 'S' (Sport bikes and general
accessories). This categorical feature helps in understanding the
primary segmentation of
the product catalog. A significant observation relates to products
with missing 'product_line' values (NaN). Out of 226 such products,
the vast majority
are not sellable (182 'Purchased & Not Sold' and 27 'Manufactured &
Not Sold'). A smaller number of non-sellable products (3 'Purchased &
Sold' and 14
'Manufactured & Sold') also have missing 'product_line'. This
indicates that 'product_line' is primarily assigned to products that
are part of the core,
sellable product offerings, especially finished goods or key
components directly associated with a bike type. Crucially, it is
observed that all products
with an unassigned 'product_line' also have unassigned 'size' values
(NaN in both 'size' and 'size_unit_measure_code'). This strong
correlation suggests
that products without a defined product line are often generic
components, raw materials, or accessories for which specific sizing is
not relevant or
tracked.
"""

# Do not modify this code
print_tile(size="h3", key='feature_13_insights',
value=feature_13_insights)

```

<IPython.core.display.HTML object>

B.14 Explore Feature of Interest `class`

```

df['class'].value_counts()

class
L      97
H      82
M      68
Name: count, dtype: int64

for c in ['L', 'H', 'M']:
    print(c)
    print(list(df[(df['class'] == c) & (df['name'].notna())]
['name'].sample(5, random_state=42)))

L
['LL Road Frame - Red, 60', 'LL Touring Frame - Yellow, 54', 'LL
Crankarm', 'Mountain-500 Black, 40', 'Road-650 Red, 60']

```

```

H
['HL Mountain Frame - Black, 42', 'HL Headset', 'HL Mountain
Seat/Saddle', 'Mountain-100 Black, 48', 'HL Mountain Tire']
M
['Road-550-W Yellow, 44', 'ML Road Frame - Red, 52', 'Mountain-400-W
Silver, 40', 'ML Crankset', 'Touring-2000 Blue, 50']

df[df['class'].isna()].groupby('is_manufactured')
['is_sellable'].value_counts()

is_manufactured  is_sellable
0.0              0.0          171
                1.0           62
1.0              0.0           20
                1.0            4
Name: count, dtype: int64

df[df['class'].isna()].describe(include='all').T

{"summary": "{\n  \"name\": \"df[df['class']]\", \n  \"rows\": 23, \n
\"fields\": [\n    {\n      \"column\": \"count\", \n
\"properties\": {\n        \"dtype\": \"date\", \n        \"min\":
\"0\", \n        \"max\": \"257\", \n        \"num_unique_values\": 11, \n
n      \"samples\": [\n        232.0, \n        \"257\", \n
n      \"61\", \n        ], \n        \"semantic_type\": \"\", \n
\"description\": \"\", \n        }, \n        {\n      \"column\":
\"unique\", \n        \"properties\": {\n        \"dtype\": \"date\", \n
\"min\": 0, \n        \"max\": 257, \n        \"num_unique_values\":
11, \n        \"samples\": [\n        5, \n        257, \n
3\n        ], \n        \"semantic_type\": \"\", \n
\"description\": \"\", \n        }, \n        {\n      \"column\":
\"top\", \n        \"properties\": {\n        \"dtype\": \"category\", \n
\"num_unique_values\": 11, \n        \"samples\": [\n        \"M\", \n
\"0ebd0784-173e-45b5-b534-a953030f49f0\", \n        \"2008-04-30
00:00:00\", \n        ], \n        \"semantic_type\": \"\", \n
\"description\": \"\", \n        }, \n        {\n      \"column\":
\"freq\", \n        \"properties\": {\n        \"dtype\": \"date\", \n
\"min\": \"1\", \n        \"max\": \"191\", \n
\"num_unique_values\": 8, \n        \"samples\": [\n        \"8\", \n
\"35\", \n        \"1\", \n        ], \n        \"semantic_type\":
\"\", \n        \"description\": \"\", \n        }, \n        {\n
\"column\": \"mean\", \n        \"properties\": {\n        \"dtype\":
\"date\", \n        \"min\": 0.0933852140077821, \n        \"max\":
709.7898832684825, \n        \"num_unique_values\": 9, \n
\"samples\": [\n        16.31408560311284, \n
0.25680933852140075, \n        54.45454545454545\n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\", \n
n      }, \n        {\n      \"column\": \"std\", \n        \"properties\": {\n
\"dtype\": \"date\", \n        \"min\": 0.2915392532520632, \n
\"max\": 411.3232868480726, \n        \"num_unique_values\": 9, \n

```

```

{"samples\":[\n          38.91028128763996,\n          0.4377257616504321,\n          46.014105468844036\n        ],\n  "semantic_type\":"\", \n  "description\":"\", \n  "column\":"\", \n  "properties\":{\n    "dtype\":"date",\n    "min\":0.0,\n    "max\":88.0,\n    "num_unique_values\":5,\n    "samples\":[\n      4.0,\n      88.0,\n      3.0\n    ],\n    "semantic_type\":"\", \n    "description\":"\", \n    "column\":"\", \n    "properties\":{\n      "dtype\":"date",\n      "min\":0.0,\n      "max\":500.0,\n      "num_unique_values\":5,\n      "samples\":[\n        500.0,\n        266.0,\n        375.0\n      ],\n      "semantic_type\":"\", \n      "description\":"\", \n      "column\":"\", \n      "properties\":{\n        "dtype\":"date",\n        "min\":0.0,\n        "max\":1000.0,\n        "num_unique_values\":5,\n        "samples\":[\n          1000.0,\n          317.0,\n          750.0\n        ],\n        "semantic_type\":"\", \n        "description\":"\", \n        "column\":"75%",\n        "properties\":{\n          "dtype\":"date",\n          "min\":0.0,\n          "max\":1000.0,\n          "num_unique_values\":8,\n          "samples\":[\n            1.0,\n            1.8663,\n            0.0\n          ],\n          "semantic_type\":"\", \n          "description\":"\", \n          "column\":"\", \n          "properties\":{\n            "dtype\":"date",\n            "min\":1.0,\n            "max\":1000.0,\n            "num_unique_values\":7,\n            "samples\":[\n              1.0,\n              1000.0,\n              245.01\n            ],\n            "semantic_type\":"\", \n            "description\":"\", \n            "column\":"\" }\n        ]\n      },\n    "type":"dataframe"}

```

```

temp = df[(df['class'].isna()) & ((df['size'].notna()) |
(df['weight'].notna()))]
display(temp['size'].value_counts())
display(temp['weight'].value_counts())

```

```

size
M      11
L      11
S       9
XL      3
70      1
Name: count, dtype: int64

```

```

weight
317.0    2
400.0    2
88.0     1
460.0    1
215.0    1
Name: count, dtype: int64

```

```
temp[temp['size'] == '70']['name']
```

```
795      Hydration Pack - 70 oz.
```

```
Name: name, dtype: object
```

```
temp[temp['weight'].notna()]['name']
```

```
365      Front Brakes
```

```
424      HL Mountain Rim
```

```
446      Front Derailleur
```

```
636      Rear Brakes
```

```
638      Touring Rim
```

```
768      Rear Derailleur
```

```
811      HL Road Rim
```

```
Name: name, dtype: object
```

```
feature_14_insights = """
```

```
The 'class' feature categorizes products into three distinct classes:  
'L', 'H', and 'M', which likely represent different quality tiers or  
product ranges
```

```
(e.g., Low, High, and Medium). This categorical feature further  
segments the product catalog. There are 257 products with missing  
'class' values. Of these,
```

```
171 are 'Purchased & Not Sold', 62 are 'Purchased & Sold', 20 are  
'Manufactured & Not Sold', and 4 are 'Manufactured & Sold'. This  
distribution suggests
```

```
that while 'class' is primarily assigned to sellable products, a  
substantial number of sellable products (62 + 4 = 66) also lack a  
defined class. Upon
```

```
closer inspection, products with missing 'class' values but with a  
defined 'size' are predominantly clothing items (except one exception  
on hydration pack
```

```
- 'Hydration Pack - 70 oz.'), where size is a relevant attribute but  
'class' might not be explicitly assigned in the same way as for  
bicycle components.
```

```
Similarly, products with missing 'class' but with a defined 'weight'  
are typically components like brakes, rims, and derailleurs. This  
indicates that
```

```
'class' is not consistently assigned across all types of products, and  
its absence is not always indicative of non-sellable or non-essential  
items. This
```

```
suggests that the absence of 'class' could be due to a combination of  
factors, including the type of product (e.g., accessories vs. main  
components) and
```

```
potential data entry inconsistencies.
```

```
"""
```

```
# Do not modify this code
```

```
print_tile(size="h3", key='feature_14_insights',  
value=feature_14_insights)
```

```
<IPython.core.display.HTML object>
```

B.15 Explore Feature of Interest style

```
df['style'].value_counts()

style
U      176
W       28
M        7
Name: count, dtype: int64

for style in ['U', 'W', 'M']:
    print(style)
    print(list(df[(df['style'] == style) & (df['name'].notna())]
['name'].sample(5, random_state=42)))

U
['Touring-3000 Blue, 50', 'HL Touring Frame - Blue, 46', 'HL Touring
Frame - Yellow, 46', 'HL Road Frame - Red, 62', 'Mountain-300 Black,
44']
W
['Road-550-W Yellow, 44', 'Road-550-W Yellow, 48', 'Road-350-W Yellow,
40', 'Road-550-W Yellow, 38', "Women's Tights, S"]
M
["Men's Sports Shorts, L", "Men's Sports Shorts, M", "Men's Bib-
Shorts, M", "Men's Sports Shorts, S", "Men's Sports Shorts, XL"]

df[df['style'].isna()].describe(include='all').T

{"summary": "{\\n  \\\"name\\\": \\\"df[df['style']]\\\",\\n  \\\"rows\\\": 23,\\n
\\\"fields\\\": [\\n    {\\n      \\\"column\\\": \\\"count\\\",\\n
\\\"properties\\\": {\\n        \\\"dtype\\\": \\\"date\\\",\\n        \\\"min\\\":
\\\"0\\\",\\n        \\\"max\\\": \\\"293\\\",\\n        \\\"num_unique_values\\\": 12,\\n
\\n        \\\"samples\\\": [\\n          \\\"71\\\",\\n          \\\"67\\\",\\n
\\\"293\\\",\\n          ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\":
\\\"unique\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"date\\\",\\n
\\\"min\\\": 0,\\n        \\\"max\\\": 293,\\n        \\\"num_unique_values\\\": 9,\\n
\\n        \\\"samples\\\": [\\n          4,\\n          23,\\n          1\\n
\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n
\\n      },\\n    {\\n      \\\"column\\\": \\\"top\\\",\\n      \\\"properties\\\": {\\n
\\n        \\\"dtype\\\": \\\"category\\\",\\n        \\\"num_unique_values\\\": 11,\\n
\\n        \\\"samples\\\": [\\n          \\\"70\\\",\\n          \\\"0ebd0784-173e-
45b5-b534-a953030f49f0\\\",\\n          \\\"2008-04-30 00:00:00\\\"\\n
\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\":
\\\"freq\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"date\\\",\\n
\\\"min\\\": \\\"1\\\",\\n        \\\"max\\\": \\\"209\\\",\\n
\\\"num_unique_values\\\": 7,\\n        \\\"samples\\\": [\\n          \\\"1\\\",\\n
```

```

\"14\", \n          \"209\" \n          ], \n          \"semantic_type\":
\"\", \n          \"description\": \"\" \n          }, \n          { \n
\"column\": \"mean\", \n          \"properties\": { \n          \"dtype\":
\"date\", \n          \"min\": 0.2150170648464164, \n          \"max\":
752.1365187713311, \n          \"num_unique_values\": 9, \n
\"samples\": [ \n          31.7118771331058, \n
0.28668941979522183, \n          83.96428571428571 \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
}, \n          { \n          \"column\": \"std\", \n          \"properties\": { \n
\"dtype\": \"date\", \n          \"min\": 0.4115370658362484, \n
\"max\": 311.12624920220685, \n          \"num_unique_values\": 9, \n
\"samples\": [ \n          68.9288928280335, \n
0.452988889523255, \n          27.348463725744224 \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
}, \n          { \n          \"column\": \"min\", \n          \"properties\": { \n
\"dtype\": \"date\", \n          \"min\": 0.0, \n          \"max\": 88.0, \n
\"num_unique_values\": 5, \n          \"samples\": [ \n          4.0, \n
88.0, \n          3.0 \n          ], \n          \"semantic_type\": \"\", \n
\"description\": \"\" \n          } \n          }, \n          { \n          \"column\":
\"25%\", \n          \"properties\": { \n          \"dtype\": \"date\", \n
\"min\": 0.0, \n          \"max\": 500.0, \n          \"num_unique_values\":
5, \n          \"samples\": [ \n          500.0, \n          215.0, \n
375.0 \n          ], \n          \"semantic_type\": \"\", \n
\"description\": \"\" \n          } \n          }, \n          { \n          \"column\":
\"50%\", \n          \"properties\": { \n          \"dtype\": \"date\", \n
\"min\": 0.0, \n          \"max\": 1000.0, \n          \"num_unique_values\":
5, \n          \"samples\": [ \n          1000.0, \n
400.0, \n          750.0 \n          ], \n          \"semantic_type\": \"\", \n
\"description\": \"\" \n          } \n          }, \n          { \n          \"column\":
\"75%\", \n          \"properties\": { \n          \"dtype\":
\"date\", \n          \"min\": 0.0, \n          \"max\": 1000.0, \n
\"num_unique_values\": 8, \n          \"samples\": [ \n          1.0, \n
10.8423, \n          0.0 \n          ], \n          \"semantic_type\": \"\", \n
\"description\": \"\" \n          } \n          }, \n          { \n          \"column\":
\"max\", \n          \"properties\": { \n          \"dtype\":
\"date\", \n          \"min\": 1.0, \n          \"max\": 1050.0, \n
\"num_unique_values\": 7, \n          \"samples\": [ \n          1.0, \n
1000.0, \n          404.99 \n          ], \n          \"semantic_type\":
\"\", \n          \"description\": \"\" \n          } \n          } \n
n} \", \"type\": \"dataframe\"}

```

```
df[(df['style'].isna()) & (df['size'].notna())]['name']
```

```

795      Hydration Pack - 70 oz.
Name: name, dtype: object

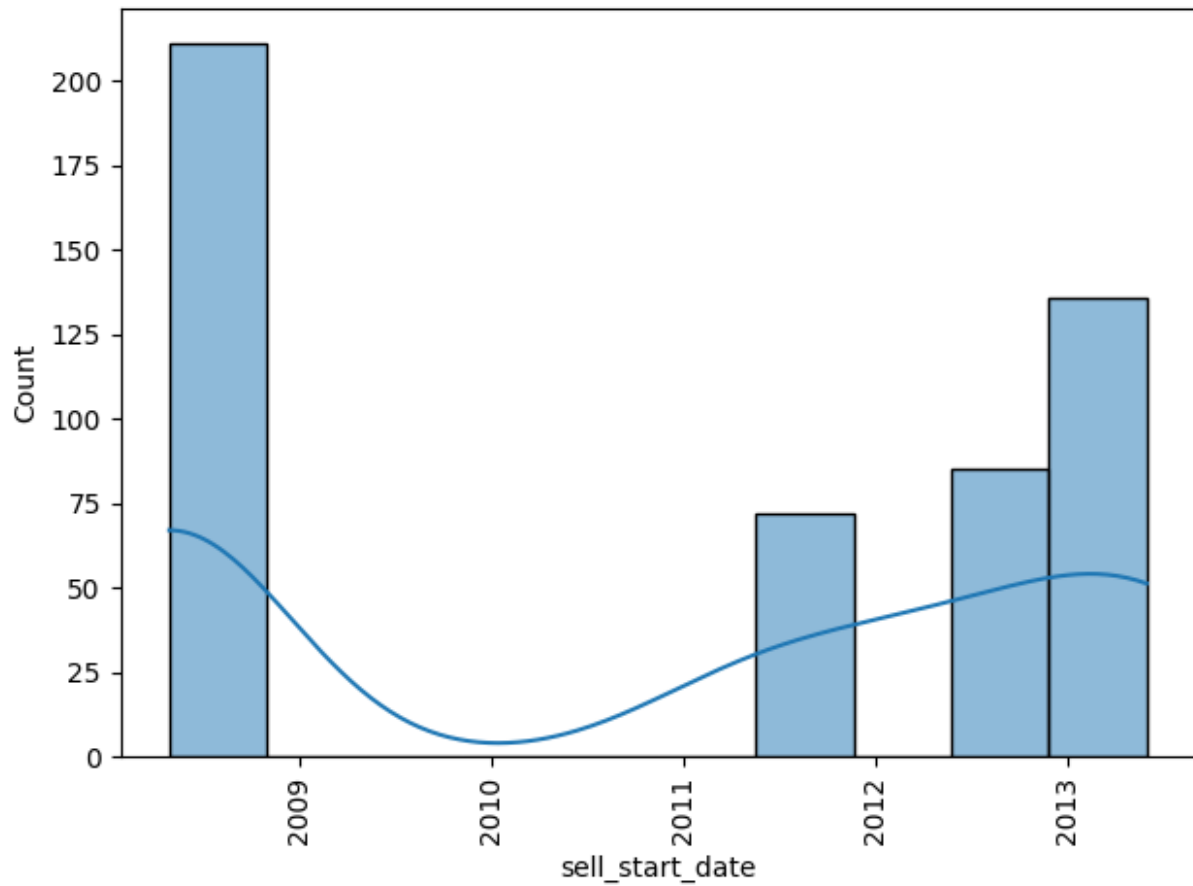
```

```
df[df['name'] == 'Hydration Pack - 70 oz.'].T
```

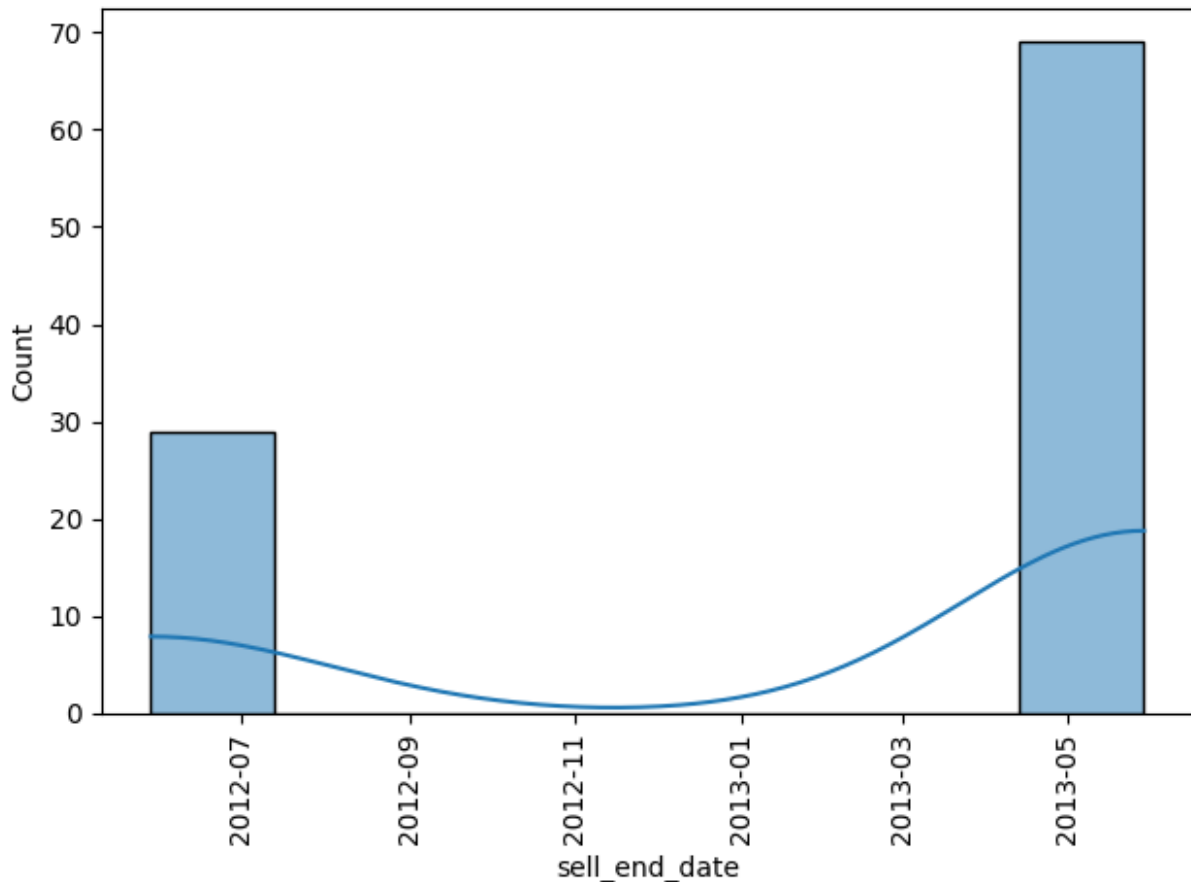
```

{"summary": "{ \n  \"name\": \"df[df['name'] == 'Hydration Pack - 70
oz\", \n  \"rows\": 23, \n  \"fields\": [ \n    { \n      \"column\":

```

```
df['sell_end_date'] = pd.to_datetime(df['sell_end_date'])
sns.histplot(df['sell_end_date'], bins='auto', kde=True)
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



```
feature_16_insights = ""
```

The 'sell_start_date' and 'sell_end_date' features define the active sales period for each product. The 'sell_start_date' shows that the majority of products began selling in 2008. This indicates a continuous product lifecycle management across these years. The 'sell_end_date' is available for only 98 products, with most of these products ending their sales period in 2013, and few products ending in 2012. The remaining 406 products have no 'sell_end_date', suggesting they are still actively sold or their sales end date was not recorded. In contrast, the 'discontinued_date' column is entirely null. This implies a clear distinction between 'sell_end_date' and 'discontinued_date'. 'sell_end_date' likely marks the point at which a product is no longer actively marketed or sold through regular channels, possibly indicating its phasing out or replacement by newer models. 'discontinued_date', if it were populated, would typically signify the complete cessation of a product's existence, including manufacturing, support, and availability of spare

```
parts.
"""

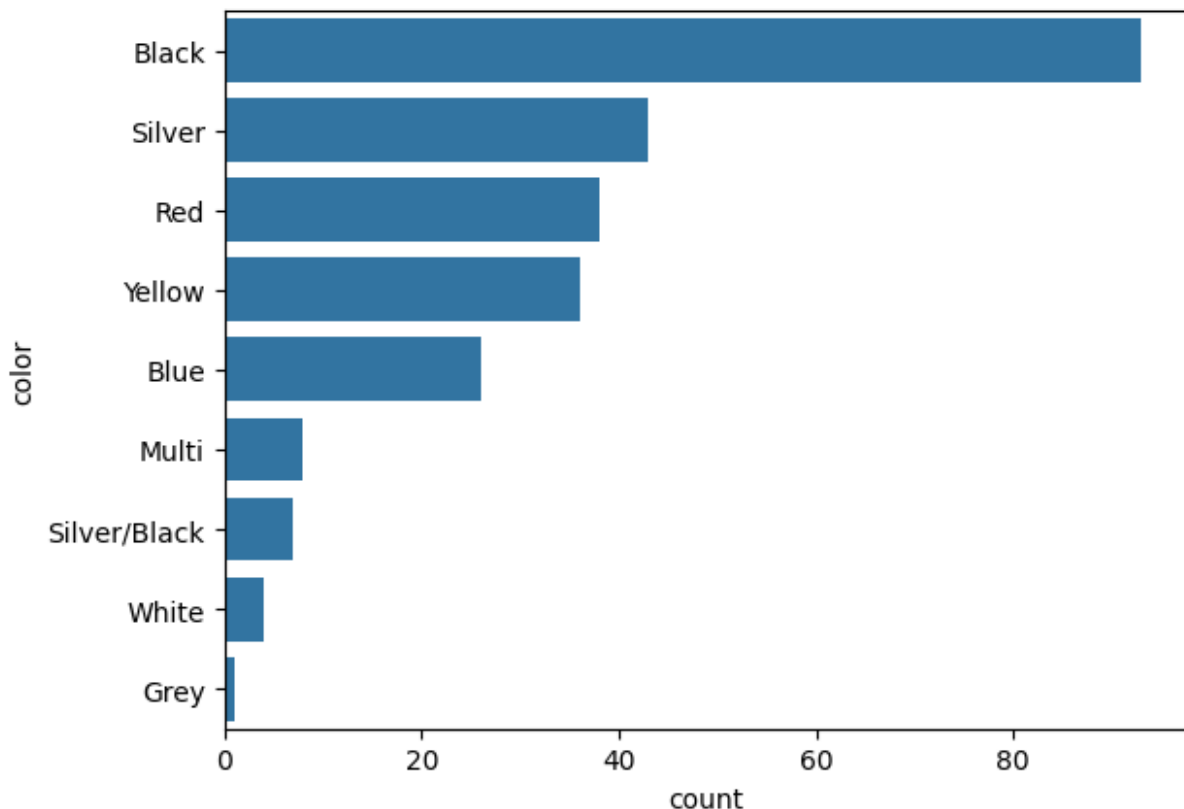
# Do not modify this code
print_tile(size="h3", key='feature_16_insights',
value=feature_16_insights)

<IPython.core.display.HTML object>
```

B.16 Explore Feature of Interest color

```
sns.barplot(y=df['color'].value_counts().index,
x=df['color'].value_counts())

<Axes: xlabel='count', ylabel='color'>
```



```
df[df['color'].isna()].describe(include='all').T

{"summary": "{\n  \"name\": \"df[df['color']]\", \n  \"rows\": 23, \n  \"fields\": [\n    {\n      \"column\": \"count\", \n      \"properties\": {\n        \"dtype\": \"date\", \n        \"min\": \"0\", \n        \"max\": \"248\", \n        \"num_unique_values\": 10, \n        \"samples\": [\n          \"48\", \n          \"50\", \n          \"222.0\" \n        ], \n        \"semantic_type\": \"\", \n
```

```

\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "unique\",\n      \ "properties\": {\n        \ "dtype\": \ "date\",\n
\ "min\": 0,\n        \ "max\": 248,\n        \ "num_unique_values\": 8,\n
n        \ "samples\": [\n          14,\n          1,\n          248\n
],\n        \ "semantic_type\": \ "\",\n        \ "description\": \ "\n
}\n    },\n    {\n      \ "column\": \ "top\",\n      \ "properties\": {\n
n        \ "dtype\": \ "category\",\n        \ "num_unique_values\": 7,\n
\ "samples\": [\n        \ "776904bb-c44c-46c4-9f48-84aa4f194272\",\n
\ "4f0705e2-0929-407d-aebe-7d39f2cd831c\",\n        \ "M\\"\n
n        ],\n        \ "semantic_type\": \ "\",\n
\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "freq\",\n      \ "properties\": {\n        \ "dtype\": \ "date\",\n
\ "min\": \ "1\",\n        \ "max\": \ "21\",\n
\ "num_unique_values\": 5,\n        \ "samples\": [\n          \ "11\",\n
\ "21\",\n          \ "10\\"\n        ],\n        \ "semantic_type\":
\ "\",\n        \ "description\": \ "\n      }\n    },\n    {\n
\ "column\": \ "mean\",\n      \ "properties\": {\n        \ "dtype\":
\ "date\",\n        \ "min\": \ "1970-01-01 00:00:00\",\n        \ "max\":
\ "2013-05-29 00:00:00\",\n        \ "num_unique_values\": 11,\n
\ "samples\": [\n          87.94,\n          0.1774193548387097,\n
\ "2009-04-12 16:50:19.354838784\\"\n        ],\n        \ "semantic_type\": \ "\",\n
\ "description\": \ "\n      }\n    },\n    {\n      \ "column\": \ "min\",\n
\ "properties\": {\n        \ "dtype\": \ "date\",\n        \ "min\": \ "1970-01-01 00:00:00\",\n
\ "max\": \ "2013-05-29 00:00:00\",\n        \ "num_unique_values\": 7,\n
\ "samples\": [\n          0.0,\n          4.0,\n          \ "2008-04-30
00:00:00\\"\n        ],\n        \ "semantic_type\": \ "\",\n
\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "25%\",\n      \ "properties\": {\n        \ "dtype\": \ "date\",\n
\ "min\": \ "1970-01-01 00:00:00\",\n        \ "max\": \ "2013-05-29
00:00:00\",\n        \ "num_unique_values\": 7,\n        \ "samples\":
[\n          0.0,\n          500.0,\n          \ "2008-04-30
00:00:00\\"\n        ],\n        \ "semantic_type\": \ "\",\n
\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "50%\",\n      \ "properties\": {\n        \ "dtype\": \ "date\",\n
\ "min\": \ "1970-01-01 00:00:00\",\n        \ "max\": \ "2013-05-29
00:00:00\",\n        \ "num_unique_values\": 7,\n        \ "samples\":
[\n          0.0,\n          1000.0,\n          \ "2008-04-30
00:00:00\\"\n        ],\n        \ "semantic_type\": \ "\",\n
\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "75%\",\n      \ "properties\": {\n        \ "dtype\": \ "date\",\n
\ "min\": \ "1970-01-01 00:00:00\",\n        \ "max\": \ "2013-05-29
00:00:00\",\n        \ "num_unique_values\": 7,\n        \ "samples\":
[\n          0.0,\n          1000.0,\n          \ "2008-04-30
00:00:00\\"\n        ],\n        \ "semantic_type\": \ "\",\n
\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "max\",\n      \ "properties\": {\n        \ "dtype\": \ "date\",\n
\ "min\": \ "1970-01-01 00:00:00.000000001\",\n        \ "max\": \ "2013-
05-30 00:00:00\",\n        \ "num_unique_values\": 9,\n

```

```

\"samples\": [\n          \"2013-05-30 00:00:00\", \n          1000.0, \n          229.49 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n          }, \n          { \n          \"column\": \"std\", \n          \"properties\": { \n          \"dtype\": \"date\", \n          \"min\": 0.38279574783838105, \n          \"max\": 302.1311059320738, \n          \"num_unique_values\": 9, \n          \"samples\": [\n          42.30992496760068, \n          0.4020159464615197, \n          22.653026107713178 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n          } \n          ] \n          }, {\"type\": \"dataframe\"}

```

```

df[(df['size'].isna()) &
(df['color'].notna())].describe(include='all').T

```

```

{"summary": "{\n  \"name\": \"df[(df['size'])\", \n  \"rows\": 23, \n  \"fields\": [\n    {\n      \"column\": \"count\", \n      \"properties\": {\n        \"dtype\": \"date\", \n        \"min\": \"0\", \n        \"max\": \"45\", \n        \"num_unique_values\": 11, \n        \"samples\": [\n          \"0\", \n          \"45\", \n          \"1\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"unique\", \n        \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": 0, \n          \"max\": 45, \n          \"num_unique_values\": 9, \n          \"samples\": [\n            4, \n            9, \n            0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n        {\n          \"column\": \"top\", \n          \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 9, \n            \"samples\": [\n              \"L\", \n              \"f9a7e077-33a4-4ef3-b979-beaf4bf163de\", \n              \"8c4f020f-f9c1-492b-8395-1372b7bda7a7\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n          }, \n          {\n            \"column\": \"freq\", \n            \"properties\": {\n              \"dtype\": \"date\", \n              \"min\": \"1\", \n              \"max\": \"22\", \n              \"num_unique_values\": 6, \n              \"samples\": [\n                \"1\", \n                \"14\", \n                \"8\" \n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\" \n            }, \n            {\n              \"column\": \"mean\", \n              \"properties\": {\n                \"dtype\": \"date\", \n                \"min\": \"1970-01-01 00:00:00\", \n                \"max\": \"2013-05-29 00:00:00\", \n                \"num_unique_values\": 11, \n                \"samples\": [\n                  75.02941176470588, \n                  0.4222222222222222, \n                  \"2011-08-28 18:08:00\" \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\" \n              }, \n              {\n                \"column\": \"min\", \n                \"properties\": {\n                  \"dtype\": \"date\", \n                  \"min\": \"1970-01-01 00:00:00\", \n                  \"max\": \"2013-05-29 00:00:00\", \n                  \"num_unique_values\": 7, \n                  \"samples\": [\n                    0.0, \n                    4.0, \n                    \"2008-04-30 00:00:00\" \n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\" \n                }, \n                {\n                  \"column\": \"25%\", \n                  \"properties\": {\n                    \"dtype\": \"date\", \n                    \"min\": \"1970-01-01 00:00:00\", \n                    \"max\": \"2013-05-29 00:00:00\", \n                    \"num_unique_values\": 10, \n                    \"samples\": [\n                      \"2011-

```

```

05-31 00:00:00\",\\n          1.0,\\n          7.95445\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n          }\\n
n      },\\n      {\\n          \\\"column\\\": \\\"50%\\\",\\n          \\\"properties\\\": {\\n
\\\"dtype\\\": \\\"date\\\",\\n          \\\"min\\\": \\\"1970-01-01 00:00:00\\\",\\n
\\\"max\\\": \\\"2013-05-29 00:00:00\\\",\\n          \\\"num_unique_values\\\": 10,\\n
n          \\\"samples\\\": [\\n          \\\"2012-05-30 00:00:00\\\",\\n
1.0,\\n          35.9596\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
n          \\\"description\\\": \\\"\\\"\\n          }\\n      },\\n      {\\n
\\\"column\\\": \\\"75%\\\",\\n          \\\"properties\\\": {\\n          \\\"dtype\\\":
\\\"date\\\",\\n          \\\"min\\\": \\\"1970-01-01 00:00:00.000000001\\\",\\n
\\\"max\\\": \\\"2013-05-30 00:00:00\\\",\\n          \\\"num_unique_values\\\": 9,\\n
\\\"samples\\\": [\\n          \\\"2013-05-30 00:00:00\\\",\\n          500.0,\\n
209.025\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n      },\\n      {\\n          \\\"column\\\":
\\\"max\\\",\\n          \\\"properties\\\": {\\n          \\\"dtype\\\": \\\"date\\\",\\n
\\\"min\\\": \\\"1970-01-01 00:00:00.000000001\\\",\\n          \\\"max\\\": \\\"2013-
05-30 00:00:00\\\",\\n          \\\"num_unique_values\\\": 9,\\n
\\\"samples\\\": [\\n          \\\"2013-05-30 00:00:00\\\",\\n          1000.0,\\n
n          404.99\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n      },\\n      {\\n          \\\"column\\\":
\\\"std\\\",\\n          \\\"properties\\\": {\\n          \\\"dtype\\\": \\\"date\\\",\\n
\\\"min\\\": 0.43461349368017643,\\n          \\\"max\\\": 328.89724483616266,\\n
\\\"num_unique_values\\\": 9,\\n          \\\"samples\\\": [\\n
116.44963210005164,\\n          0.43461349368017643,\\n
34.64843806363505\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n      }\\n      ]\\n}\\\", \"type\": \"dataframe\"}

```

```

df[(df['color'].isna()) & (df['name'].notna())]['name'].sample(25,
random_state=42)

```

```

329          Cable Lock
60          Flat Washer 6
473    External Lock Washer 8
759          Lock Washer 5
299    HL Touring Seat/Saddle
710          HL Road Handlebars
250          Lock Nut 9
247          Lock Nut 20
517          Bearing Ball
750          ML Road Rim
758    Thin-Jam Lock Nut 15
616          Adjustable Race
239          Metal Tread Plate
97    HL Touring Handlebars
349          Lock Washer 8
343          Flat Washer 3
182    Thin-Jam Lock Nut 11
214          Road End Caps
376          Lock Nut 22
685    Headset Ball Bearings

```

```
66          Blade
525      Paint - Black
401          LL Nipple
749          Seat Post
244  Internal Lock Washer 8
Name: name, dtype: object
```

```
feature_17_insights = """
The 'color' feature categorizes products into nine distinct colors,
with 'Black' being the most prevalent. A significant portion of the
dataset, 248 out of
504 products, have missing 'color' values. These products are
predominantly accessory items, such as 'Lock Nut', 'Washer', 'Cable
Lock', 'Bearing Ball',
and 'Seat Post', for which color might not be a critical attribute or
is not explicitly tracked. All products with missing 'color' also have
missing 'size'
and 'style'. But, some products without a defined 'size' or 'style' do
have a 'color' assigned. This indicates that while 'color' is a
crucial descriptive
attribute for many products, its absence is often systematic,
reflecting the nature of certain components or non-consumer-facing
items.
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_17_insights',
value=feature_17_insights)
```

```
<IPython.core.display.HTML object>
```